

C 语言基础教程

Kuladmir Present

V 1.18

2026-09-16

C 语言基础教程

目录/Contents

1.基础知识.....	1
2.ASCII 码表.....	1
3.常用 C 语言头文件及其内置函数.....	2
4.关键字	12
5.操作符	12
6.基本语法.....	15
7.判断.....	15
8.循环.....	15
9.函数.....	16
10.数组	17
11.结构体.....	19
12.枚举	21
13.联合体.....	22
14.指针	22
15.数据存储	28
16.内存动态分布.....	29
17.文件操作	30
18.宏的定义和使用	34
19.预处理.....	34
EX.经典例题	36
20.简单界面	47

注：本教程为总结性教程，1-19 章详情参考：

https://www.bilibili.com/video/BV1q54y1q79w/?vd_source=6a66772e6a688ab2ca4e733f22547766

另：本教程仅用于学习和参考，不做他用！

阅读说明：加红为较重要定义或提示，加红加粗为重要知识点，加蓝加粗表示重要关键字，绿色表示举例用法，补充：内容。

1.基础知识

1) 电信号：正电为 1，负电为 0，每个 1 和 0 存储在一个格子，该格子称为一个 bit，8 个 bit 是一个字节 byte，1Gb=1024Mb=1024*1024Kb=1024^3 字节。

2) 计算机存储整数：存储其补码。对于 int 来说，其占用 4 个字节，32 个 bit 位。以 8 字节为例。最高位表示正负，1 负 0 正。换算方法：反码（符号位不变，其他位 1、0 互换），补码（反码+1），正数三码合一。举例：原

（00000010）反（01111101）补（01111110）。在进行左移、右移、按位与、按位异或、按位或、按位取反操作时，都需要使用其补码进行计算，在输出时需要换算成其原码得到数值。

3) 存储：读取速度 硬盘<内存<寄存器。

4) 内存可简单分为：栈区（储存局部变量、函数形参，函数的调用），堆区（动态开辟空间），静态区（全局变量、static 修饰的变量）。栈区使用时，往往先使用高地址处的空间。而数组是从低到高使用地址，因此越界访问可能导致死循环。

5) 调试的步骤：发现错误->对错误定位->确定错误原因->解决问题->再次测试
调试快捷键：F5（启动调试，和 F9 一起使用，设置断点以后按 F5），F9（设置断点：使代码执行至断点处停止），F10（逐过程：按一次执行一步），F11（逐语句：可以关注到更多细节）。

2.ASCII 码表

ASCII 字符代码表 一																							
高四位 低四位		ASCII 非打印控制字符										ASCII 打印字符											
		0000					0001					0010	0011	0100	0101	0110	0111						
		0					1					2	3	4	5	6	7						
	+进制	字符	ctrl	代码	字符解释	+进制	字符	ctrl	代码	字符解释	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符	ctrl		
0000	0	0	BLANK NULL	^@	NUL	空	16	▶	^P	DLE	数据链路转意	32		48	0	64	@	80	P	96	`	112	p
0001	1	1	☺	^A	SOH	头标开始	17	◀	^Q	DC1	设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q
0010	2	2	☹	^B	STX	正文开始	18	↕	^R	DC2	设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r
0011	3	3	♥	^C	ETX	正文结束	19	!!	^S	DC3	设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s
0100	4	4	♦	^D	EOT	传输结束	20	¶	^T	DC4	设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t
0101	5	5	♣	^E	ENQ	查询	21	§	^U	NAK	反确认	37	%	53	5	69	E	85	U	101	e	117	u
0110	6	6	♠	^F	ACK	确认	22	■	^V	SYN	同步空闲	38	&	54	6	70	F	86	V	102	f	118	v
0111	7	7	●	^G	BEL	震铃	23	↑	^W	ETB	传输块结束	39	'	55	7	71	G	87	w	103	g	119	w
1000	8	8	▣	^H	BS	退格	24	↑	^X	CAN	取消	40	(	56	8	72	H	88	X	104	h	120	x
1001	9	9	○	^I	TAB	水平制表符	25	↓	^Y	EM	媒体结束	41	)	57	9	73	I	89	Y	105	i	121	y
1010	A	10	◻	^J	LF	换行/新行	26	→	^Z	SUB	替换	42	*	58	:	74	J	90	Z	106	j	122	z
1011	B	11	♂	^K	VT	垂直制表符	27	←	^[	ESC	转意	43	+	59	;	75	K	91	[	107	k	123	{
1100	C	12	♀	^L	FF	换页/新页	28	└	^\	FS	文件分隔符	44	,	60	<	76	L	92	\	108	l	124	
1101	D	13	🎵	^M	CR	回车	29	↔	^]	GS	组分隔符	45	-	61	=	77	M	93	]	109	m	125	}
1110	E	14	🎵	^N	SO	移出	30	▲	^_	RS	记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~
1111	F	15	☼	^O	SI	移入	31	▼	^-	US	单元分隔符	47	/	63	?	79	O	95	_	111	o	127	Δ

注：表中的 ASCII 字符可以用:ALT + “小键盘上的数字键”输入

\o 表示输出八进制数字，**\x** 表示输出十六进制数字；
 转义字符：**\n** 换行，**\a** 响铃，**\t** 水平制表符，等于 Tab，**\v** 垂直制表符，
\'(" ? \) 表示输出该内容，**\b** 退格，**\r** 回车，**\f** 换行，**%%** 输出 %。

3. 常用 C 语言头文件及其内置函数

#include<stdio.h>

remove	Remove file (function)
rename	Rename file (function)
tmpfile	Open a temporary file (function)
tmpnam	Generate temporary filename (function)
fclose	Close file (function)
fflush	Flush stream (function)
fopen	Open file (function)
freopen	Reopen stream with different file or mode (function)
setbuf	Set stream buffer (function)
setvbuf	Change stream buffering (function)
fread	Read block of data from stream (function)
fwrite	Write block of data to stream (function)
clearerr	Clear error indicators (function)
feof	Check end-of-file indicator (function)
ferror	Check error indicator (function)
perror	Print error message (function)
fgetc	Get character from stream (function)
fgets	Get string from stream (function)
fputc	Write character to stream (function)
fputs	Write string to stream (function)
getc	Get character from stream (function)
getchar	Get character from stdin (function)
gets	Get string from stdin (function)
putc	Write character to stream (function)
putchar	Write character to stdout (function)
puts	Write string to stdout (function)
ungetc	Unget character from stream (function)
fgetpos	Get current position in stream (function)
fseek	Reposition stream position indicator (function)
fsetpos	Set position indicator of stream (function)
ftell	Get current position in stream (function)
rewind	Set position of stream to the beginning (function)
fprintf	Write formatted data to stream (function)
fscanf	Read formatted data from stream (function)
printf	Print formatted data to stdout (function)
scanf	Read formatted data from stdin (function)
snprintf C++11	Write formatted output to sized buffer (function)
sprintf	Write formatted data to string (function)
sscanf	Read formatted data from string (function)
vfprintf	Write formatted data from variable argument list to stream (function)
vscanf C++11	Read formatted data from stream into variable argument list (function)
vprintf	Print formatted data from variable argument list to stdout (function)
vscanf C++11	Read formatted data into variable argument list (function)

vsprintf C++11	Write formatted data from variable argument list to sized buffer (function)
vsprintf	Write formatted data from variable argument list to string (function)
vsscanf C++11	Read formatted data from string into variable argument list (function)

#include<string.h>

memcpy	Copy block of memory (function)
memmove	Move block of memory (function)
strcpy	Copy string (function)
strncpy	Copy characters from string (function)

strcat	Concatenate strings (function)
strncat	Append characters from string (function)

memcmp	Compare two blocks of memory (function)
strcmp	Compare two strings (function)
strcoll	Compare two strings using locale (function)
strncmp	Compare characters of two strings (function)
strxfrm	Transform string using locale (function)

memset	Fill block of memory (function)
strerror	Get pointer to error message string (function)
strlen	Get string length (function)

memchr	Locate character in block of memory (function)
strchr	Locate first occurrence of character in string (function)
strcspn	Get span until character in string (function)
strpbrk	Locate characters in string (function)
strrchr	Locate last occurrence of character in string (function)
strspn	Get span of character set in string (function)
strstr	Locate substring (function)
strtok	Split string into tokens (function)

strcat 函数： 将一个字符串内容追加到另一个字符串后面，**但是不能追加到相同的后面。**

函数形式：**char * strcat (char * destination, const char * source);**

用法：strcat(被追加字符串，欲追加的字符串);

strncat 函数： 将一个字符串内容追加到另一个字符串后面，可以进行相同追加。

函数形式：**char * strncat (char * destination, const char * source, size_t num);**

用法：strncat(被追加字符串，欲追加的字符串，长度);

```
#include<stdio.h>
#include<string.h>
int main()
{
    char a[20];
    char b[20];
    printf("请输入一个字符串:>");
    gets_s(a);
    printf("请输入一个字符串:>");
    gets_s(b);
    strcat(a, b);
    int len = strlen(b);
    strncat(b, b, len);
    printf("a -> %s\n", a);
    printf("b -> %s\n", b);
    return 0;
}
```

```
请输入一个字符串:>kuladmir
请输入一个字符串:>pati
a -> kuladmirpati
b -> patipati
```

```
D:\Microsoft Visual 2022\Project
按任意键关闭此窗口. . .
```

strstr 函数：用于寻找一个字符串里有无要找的字符串（即子串），返回一个指针，如果无则返回空指针。

函数形式：**const char * strstr (const char * str1, const char * str2);**

用法：**strstr**(被查找的字符串，寻找的字符串);

说明：**strstr 函数（能找到）**会返回被查字符串中子串首次出现的地址。

```
#include<stdio.h>
#include<string.h>
int main()
{
    char a[20], b[20];
    printf("请输入一个长度小于20的字符串:>");
    gets_s(a);
    printf("请输入一个长度小于20的字符串:>");
    gets_s(b);
    char* p = strstr(a, b);
    if (p == NULL)
        printf("未找到\n");
    else
    {
        printf("找到了 %s\n", p);
    }
    return 0;
}
```

```
请输入一个长度小于20的字符串:>kualdmirdmir
请输入一个长度小于20的字符串:>dmir
找到了 dmir
```

```
D:\Microsoft Visual 2022\Project1\x64\Release
按任意键关闭此窗口. . .
```

strlen 函数：只能用于计算字符串长度，当找到停止符‘\0’时停止，**返回无符号 int。**

函数类型：**size_t strlen (const char * str);//size_t -> unsigned int**

用法：**int a=strlen**(欲求长度的字符串);

char a[]={‘a’,‘b’,‘c’};int c=strlen(a); ->c=随机值。因为该数组中无停止符，因此计算机机会一直找下去，直到找到停止符。

char a[]="abc";int c=strlen(a);->a=3。因为该数组内容为：**a,b,,\0**，因此为**3**。


```
#include<stdio.h>
#include<string.h>
int main()
{
    char a[10] = { 'a', 'b', 'c' };
    char b[10] = "kuladmir";
    int m = strlen(a);
    int n = strlen(b);
    printf("m=%d n=%d\n", m, n);
    return 0;
}
```

```
m=3 n=8
D:\Microsoft Visual 2
按任意键关闭此窗口. .
```

strcpy 函数：用于将一个字符串内容拷贝到另一个字符串。

函数形式：**char * strcpy (char * destination, const char * source);**

用法：strcpy(被拷贝的字符串, 拷贝内容的字符串);

```
#include<stdio.h>
#include<string.h>
int main()
{
    char a[20];
    char b[10];
    printf("请输入一个字符串:>");
    gets_s(a);
    printf("请输入一个字符串:>");
    gets_s(b);
    strcpy(a, b);
    printf("a -> %s\n", a);
    printf("b -> %s\n", b);
    return 0;
}
```

```
请输入一个字符串:>kulapati
请输入一个字符串:>admirmoon
a -> admirmoon
b -> admirmoon
D:\Microsoft Visual 2022\Project1\x64\Rel
按任意键关闭此窗口. .
```

strncpy 函数：用以拷贝字符串内容到另一个字符串，且可以控制拷贝长度。

函数形式：**char * strncpy (char * destination, const char * source, size_t num);**

用法：strncpy(被拷贝的字符串, 拷贝内容的字符串, 指定长度);

函数形式：

说明：**当指定的长度大于拷贝内容的字符串元素个数时，将在拷贝内容后补 0。**

```
#include<stdio.h>
#include<string.h>
int main()
{
    char a[20], b[20];
    printf("请输入一个长度小于20的字符串:>");
    gets_s(a);
    printf("请输入一个长度小于20的字符串:>");
    gets_s(b);
    strncpy(a, b, 6);
    printf("%s\n", a);
    return 0;
}
```

```
请输入一个长度小于20的字符串:>kuladmir
请输入一个长度小于20的字符串:>admirm
admirmir
D:\Microsoft Visual 2022\Project1\x64\Relea
按任意键关闭此窗口. .
```

strcmp 函数：用于比较两个字符串的大小（根据元素的 ASCII 码确定），第一个大返回整数，小返回负数，相等返回 0。

函数形式：**int strcmp (const char * str1, const char * str2);**

用法: `int ret=strcmp(字符串, 字符串);`

```
#include<stdio.h>
#include<string.h>
int main()
{
    char a[20];
    char b[20];
    gets_s(a);
    gets_s(b);
    int ret=strcmp(a, b);
    if (ret > 0)
        printf("a>b\n");
    else if (ret < 0)
        printf("a<b\n");
    else
        printf("a==b\n");
    return 0;
}
```

```
kuladmir
pati
a<b
D:\Microsoft Visual 2022\Project1\Release
按任意键关闭此窗口. . .
```

strncmp 函数: 用于比较两个字符串前几个字符的大小。

函数形式: **`int strncmp ( const char * str1, const char * str2, size_t num );`**

用法: `strncmp(字符串, 字符串, 比较长度);`

```
#include<stdio.h>
#include<string.h>
int main()
{
    char a[20], b[20];
    printf("请输入一个长度小于20的字符串:>");
    gets_s(a);
    printf("请输入一个长度小于20的字符串:>");
    gets_s(b);
    int ret = strncmp(a, b, 6);
    if (ret > 0)
        printf("a>b\n");
    else if (ret < 0)
        printf("a<b\n");
    else
        printf("a==b\n");
    return 0;
}
```

```
请输入一个长度小于20的字符串:>kuladmir
请输入一个长度小于20的字符串:>admirmoon
a>b
D:\Microsoft Visual 2022\Project1\x64\Release
按任意键关闭此窗口. . .
```

strtok 函数: 用于分割字符串。

函数类型: **`char * strtok ( char * str, const char * delimiters );`**

用法: `char* p=strtok(被分割字符串, 分割标志字符串);`

```
#include<stdio.h>
#include<string.h>
int main()
{
    char a[20], b[20];
    printf("请输入一个小于20的字符串:>");
    gets_s(a);
    printf("请输入一个小于20的字符串:>");
    gets_s(b);
    char* p;
    for (p = strtok(a, b); p != NULL; p = strtok(NULL, b))
        printf("%s ", p);
    return 0;
}
```

```
请输入一个小于20的字符串:>kuladmir@163.com
请输入一个小于20的字符串:>.
kuladmir 163 com
D:\Microsoft Visual 2022\Project1\x64\Release
按任意键关闭此窗口. . .
```

strerror 函数: 用于打印错误码对应的错误原因。

函数类型: **char * strerror (int errnum);**

用法: char*p=strerror(errno); (需要#include<errno.h>确保 errno 执行)

```
#include<stdio.h>
#include<string.h>
#include<errno.h>
int main()
{
    FILE* p = fopen("test.c", "r");
    if (p == NULL)
        printf("%s\n", strerror(errno));
    else
        printf("Open successfully\n");
    return 0;
}
```

```
No such file or directory
D:\Microsoft Visual 2022\Proj
按任意键关闭此窗口. . .
```

memset 函数: 用于更改字符串内容。

函数类型: **void * memset (void * ptr, int value, size_t num);**

用法: memset (欲修改的字符串, 欲更改的内容, 设置长度(字节));

```
#include<stdio.h>
#include<string.h>
int main()
{
    char a[20];
    printf("请输入一个字符串:>");
    gets_s(a);
    memset(a, '*', 3);
    printf("%s\n", a);
    return 0;
}
```

```
请输入一个字符串:>kuladmir
***admir
D:\Microsoft Visual 2022\Projec
按任意键关闭此窗口. . .
```

memcpy 函数: 用于拷贝内容, 可以拷贝各种类型内容。

函数类型: **void * memcpy (void * destination, const void * source, size_t num);**

用法: memcpy(目的地, 源头, 拷贝长度(字节));

```
#include<stdio.h>
#include<string.h>
int main()
{
    int a[5], b[5];
    int i;
    for (i = 0; i < 5; i++)
        scanf("%d", &a[i]);
    memcpy(b, a, 20);
    for (i = 0; i < 5; i++)
        printf("%d ", b[i]);
    printf("\n");
    return 0;
}
```

```
5 6 7 8 9
5 6 7 8 9
D:\Microsoft Vis
按任意键关闭此窗
```

memmove 函数: 用于重叠拷贝。

函数类型: **void * memmove (void * destination, const void * source, size_t num);**

用法: memmove(目的地,源头,拷贝长度(字节));

```
#include<stdio.h>
#include<string.h>
int main()
{
    int a[10];
    int i;
    for (i = 0; i < 10; i++)
        scanf("%d", &a[i]);
    memmove(a + 3, a, 20);
    for (i = 0; i < 10; i++)
        printf("%d ", a[i]);
    return 0;
}
```

```
1 2 3 4 5 6 7 8 9 10
1 2 3 1 2 3 4 5 9 10
D:\Microsoft Visual 2022\Pr
按任意键关闭此窗口. . .
```

memcmp 函数: 用于比较内容, 可以比较大部分类型。

函数类型: **int memcmp (const void * ptr1, const void * ptr2, size_t num);**

用法: int a=memcmp(类型 1,类型 2,比较长度 (字节));

```
#include<stdio.h>
#include<string.h>
int main()
{
    int a[5] = { 1, 2, 3, 4, 5 };
    int b[5] = { 2, 3, 4, 5, 6 };
    int c = memcmp(a, b, 20);
    if (c > 0)
        printf("a>b\n");
    else if (c < 0)
        printf("a<b\n");
    else
        printf("a==b\n");
    return 0;
}
```

```
a<b
D:\Microsoft Visual 2022\Pr
按任意键关闭此窗口. . .
```

#include<math.h>

Error and gamma functions

erf	Compute error function (function)
erfc	Compute complementary error function (function)
tgamma	Compute gamma function (function)
lgamma	Compute log-gamma function (function)

Power functions

pow	Raise to power (function)
sqrt	Compute square root (function)
cbrt	Compute cubic root (function)
hypot	Compute hypotenuse (function)

Trigonometric functions

cos	Compute cosine (function)
sin	Compute sine (function)
tan	Compute tangent (function)
acos	Compute arc cosine (function)
asin	Compute arc sine (function)
atan	Compute arc tangent (function)
atan2	Compute arc tangent with two parameters (function)

Hyperbolic functions

cosh	Compute hyperbolic cosine (function)
sinh	Compute hyperbolic sine (function)
tanh	Compute hyperbolic tangent (function)
acosh	Compute area hyperbolic cosine (function)
asinh	Compute area hyperbolic sine (function)
atanh	Compute area hyperbolic tangent (function)

Rounding and remainder functions

ceil	Round up value (function)
floor	Round down value (function)
fmod	Compute remainder of division (function)
trunc	Truncate value (function)
round	Round to nearest (function)
lround	Round to nearest and cast to long integer (function)
llround	Round to nearest and cast to long long integer (function)
rint	Round to integral value (function)
lrint	Round and cast to long integer (function)
llrint	Round and cast to long long integer (function)
nearbyint	Round to nearby integral value (function)
remainder	Compute remainder (IEC 60559) (function)
remquo	Compute remainder and quotient (function)

Floating-point manipulation functions

copysign	Copy sign (function)
nan	Generate quiet NaN (function)
nextafter	Next representable value (function)
nexttoward	Next representable value toward precise value (function)

Minimum, maximum, difference functions

<u>fdim</u>	Positive difference (function)
<u>fmax</u>	Maximum value (function)
<u>fmin</u>	Minimum value (function)

Other functions

<u>fabs</u>	Compute absolute value (function)
<u>abs</u>	Compute absolute value (function)
<u>fma</u>	Multiply-add (function)

Exponential and logarithmic functions

<u>exp</u>	Compute exponential function (function)
<u>frexp</u>	Get significand and exponent (function)
<u>ldexp</u>	Generate value from significand and exponent (function)
<u>log</u>	Compute natural logarithm (function)
<u>log10</u>	Compute common logarithm (function)
<u>modf</u>	Break into fractional and integral parts (function)
<u>exp2</u>	Compute binary exponential function (function)
<u>expm1</u>	Compute exponential minus one (function)
<u>ilogb</u>	Integer binary logarithm (function)
<u>log1p</u>	Compute logarithm plus one (function)
<u>log2</u>	Compute binary logarithm (function)
<u>logb</u>	Compute floating-point base logarithm (function)
<u>scalbn</u>	Scale significand using floating-point base exponent (function)
<u>scalbln</u>	Scale significand using floating-point base exponent (long) (function)

#include<stdlib.h>

String conversion

<u>atof</u>	Convert string to double (function)
<u>atoi</u>	Convert string to integer (function)
<u>atol</u>	Convert string to long integer (function)
<u>atoll</u>	Convert string to long long integer (function)
<u>strtod</u>	Convert string to double (function)
<u>strtof</u>	Convert string to float (function)
<u>strtol</u>	Convert string to long integer (function)
<u>strtold</u>	Convert string to long double (function)
<u>strtoll</u>	Convert string to long long integer (function)
<u>strtoul</u>	Convert string to unsigned long integer (function)
<u>strtoull</u>	Convert string to unsigned long long integer (function)

Pseudo-random sequence generation

<u>rand</u>	Generate random number (function)
<u>srand</u>	Initialize random number generator (function)

Pseudo-random sequence generation

<u>rand</u>	Generate random number (function)
<u>srand</u>	Initialize random number generator (function)

Dynamic memory management

<u>calloc</u>	Allocate and zero-initialize array (function)
<u>free</u>	Deallocate memory block (function)
<u>malloc</u>	Allocate memory block (function)
<u>realloc</u>	Reallocate memory block (function)

Environment

<u>abort</u>	Abort current process (function)
<u>atexit</u>	Set function to be executed on exit (function)
<u>at_quick_exit</u>	Set function to be executed on quick exit (function)
<u>exit</u>	Terminate calling process (function)
<u>getenv</u>	Get environment string (function)
<u>quick_exit</u>	Terminate calling process quick (function)
<u>system</u>	Execute system command (function)
<u>_Exit</u>	Terminate calling process (function)

Searching and sorting

<u>bsearch</u>	Binary search in array (function)
<u>qsort</u>	Sort elements of array (function)

Integer arithmetics

<u>abs</u>	Absolute value (function)
<u>div</u>	Integral division (function)
<u>labs</u>	Absolute value (function)
<u>ldiv</u>	Integral division (function)
<u>llabs</u>	Absolute value (function)
<u>lldiv</u>	Integral division (function)

Multibyte characters

<u>mblen</u>	Get length of multibyte character (function)
<u>mbtowc</u>	Convert multibyte sequence to wide character (function)
<u>wctomb</u>	Convert wide character to multibyte sequence (function)

Multibyte strings

<u>mbstowcs</u>	Convert multibyte string to wide-character string (function)
<u>wcstombs</u>	Convert wide-character string to multibyte string (function)

#include<time.h>

Time manipulation

<u>clock</u>	Clock program (function)
<u>difftime</u>	Return difference between two times (function)
<u>mktime</u>	Convert tm structure to time_t (function)
<u>time</u>	Get current time (function)

Conversion

asctime	Convert tm structure to string (function)
ctime	Convert time_t value to string (function)
gmtime	Convert time_t to tm as UTC time (function)
localtime	Convert time_t to tm as local time (function)
strftime	Format time as string (function)

4.C 关键字

定义数值类型：**int** 整型 4 字节，**float** 单精度浮点 4 字节；**double** 双精度浮点 8 字节，**char** 字符 1 字节，**long** 长整型 4 字节，**short** 短整型 2 字节，**signed/unsigned** 表示有无符号：**unsigned int**；**sizeof** 求字节数：**int a,b;b=sizeof(int)或 b=sizeof(a) -> b=4**，即可求出内存划分字节数。

切断：**break**：跳出本循环，也可用于 case{}，以跳出 **switch**。**continue**：跳过其之后的语句。

判断：**if{}else{}，if{}else if{}。switch(){case 分支选项: {}default: {}}**

循环：**while(循环条件，真则执行){}。do{}while(循环条件);。for(;;){}。**

其他：**typedef** 自定义类型：**typedef unsigned int u_int**；**const** 修饰变量成为常变量，使其之后不能再改变，**extern** 引入外变量，**goto** 跳到某一位置，尽量不用，**static** 能改变量生命周期，可长可短，**enum** 枚举：**enum{a,b,c};**，**register** 寄存器关键词，建议存至寄存器，**union** 共用体，**auto** 自动（常省略）：**(auto) int a**；**return** 返回值，**struct** 结构体，**void** 空型，**volatile** 阻止编译器优化。

对应内容（输入输出）：**int - %d float - %f double - %lf 字符 - %c 字符串 - %s 地址 - %p 打印十进制无符号数 - %u**

5.操作符【a,b】 前面表示几目，后表示优先级，优先级小的先计算，在计算表达式的时候需要先注意优先级，再考虑结合方式，再看是否有输出限制。

1) **()** 括号、**[]** 下标运算、**->** 指向结构体成员运算符 **【,1】** 从左至右结合

2) **++** 自增、**--** 自减、**(类型)** 强制转换、**!** 逻辑非、**~** 按位取反（补码：1->0，0->1）、**-** 取负、***** 指针运算、**&** 取地址、**sizeof** 长度计算 **【1,2】** 从右至左结合

说明：**a++** 表示先用 **a** 后自增，**++a** 表示先自增后用 **a**；**a++** 等价于

a=a+1;a=a+2 等价于 **a+=2**，对 **+ - * /** 均适用。 **a++;** **a+=2;** **a/=2;**

sizeof（） 内部的表达式不参与运算，且其返回值为无符号数，在比较大小时，会将有符号数转化为无符号数。


```

#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    short a = 2;
    int b;
    printf("%d\n", sizeof(a = a + 2));
    printf("%d\n", a);
    return 0;
}

```

3) *乘、/除、%取余数【2,3】从左至右结合

说明：/ 除号两端若都为整型，则执行整型除法，若有一边为浮点型，则为浮点型除法。

```

#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    printf("5/2 = %d\n", 5 / 2);
    printf("5/2.0 = %f\n", 5 / 2.0);
    return 0;
}

```

%取模两端需要都为整型。

4) +加、-减【2,4】从左至右结合

5) <<左移、>>右移：将机器数向左或向右移动几个格子【2,5】从左至右结合
 $b=2(00000010); a=b<<2; a=8(00001000);$ 说明：右面操作数不能为负数，且只能作用于整数。常和按位操作进行二进制位操作。

6) <大于、<=大于等于、>小于、>=小于等于【2,6】从左至右结合

7) !=判断不等于、==判断等于【2,7】从左至右结合 $\text{if}(a!=2) \text{ if}(a==2)$

8) &按位与【2,8】从左至右结合：每位同1则1，否则为0 只能作用于整数
 $a=2(00000010); b=5(00000101); c=a\&b=0(00000000);$

9) ^按位异或【2,9】从左至右结合：每位相同为0，不同为1 只能作用于整数

$a=2(00000010); b=5(00000101); c=a\^b=7(00000111)$

10) |按位或【2,10】从左至右结合：每位有1则1，否则为0 只能作用于整数

$a=2(00000010); b=5(00000101); c=a|b=7(00000111);$

11) &&逻辑且【2,11】从左至右结合 $\text{if}(a==2 \&\& b==5)$

说明：在进行该操作计算时，若左边为0，则不执行右侧内容。判断为真则返回1，假则返回0。

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    int a = 5, b = 2;
    int c = a && b;
    printf("%d\n", c);

    int d = 2, f = 0;
    int e = f && d;
    printf("%d\n", e);

    int a1 = 2, b1 = 3, c1 = 6;
    int d1 = a1++ && b1++ && c1++;
    printf("%d %d %d %d\n", a1, b1, c1, d1);

    int a2 = 0, b2 = 5, c2 = 6;
    int d2 = a2++ && b2++ && c2++;
    printf("%d %d %d %d\n", a2, b2, c2, d2);
    return 0;
}
```

```
1
0
3 4 7 1
1 5 6 0
D:\Microsoft Visual
按任意键关闭此窗口.
```

12) ||逻辑或【2,12】从左至右结合 if(a==2 || b==5)

说明：在进行该操作计算时，若左边不为 0，则不执行右侧内容。判断为真则返回 1，假则返回 0。

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    int a = 5, b = 2;
    int c = a || b;
    printf("%d\n", c);//1

    int d = 2, f = 0;
    int e = f || d;
    printf("%d\n", e);//1

    int a1 = 2, b1 = 3, c1 = 6;
    int d1 = a1++ || b1++ || c1++;
    printf("%d %d %d %d\n", a1, b1, c1, d1);//3 3 6 1

    int a2 = 0, b2 = 5, c2 = 6;
    int d2 = a2++ || b2++ || c2++;
    printf("%d %d %d %d\n", a2, b2, c2, d2);//1 6 6 1
    return 0;
}
```

```
1
1
3 3 6 1
1 6 6 1
D:\Microsoft Visual 2022\Pro
按任意键关闭此窗口. . .
```

13) ?:条件运算【3,13】return (x>y?a:b)

说明：上例中 (x>y) 表示条件，如果成立则执行 a，否则执行 b。

14) =赋值【2,14】a=5;

15) ,逗号表达式【,15】从左至右执行，最后一个式子结果为整个的结果。

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    int a = 5, b = 6;
    int c = (a = a + b, b = b + 5, a++);
    printf("%d\n", a);
    return 0;
}
```

```
12
D:\Microsoft Vis
按任意键关闭此窗
```

隐式类型转换：整型算数运算总是至少以缺省整型类型的精度来进行的。

1) 整型提升：表达式中的字符和短整型操作数使用前被转化成普通整型。在执行语句时，先被转化为 `int` 类型计算，后进行截断（从低位开始数字字节数截断）。**整型提升的执行：对于有符号类型变量按照变量的数据类型的符号位进行提升。**如果一个数据类型为**无符号类型**，则在**整型提升时补 0**。**先提升再执行。**
例如：`char a=1;`其二进制为：`00000000000000000000000000000001`（下划线为实际存储的）

因此在之后的操作中，将发生整型提升操作：按照符号位补齐 32 位。而对于 `unsigned char a=1;`发生整型提升时则直接补 0。

2) 算数转换：如果有不同类型的操作数进行计算，则会以更高级的类型进行计算。

6.基本语法(除输出内容外，必须为英文形式)

1) 先引头文件 `#include<.....>`

2) 编写主函数大体框架 `int main(){.....;return 0;}`，其中编写所需要的内容。

3) 常量定义：`#define` 标识符 数值 `#define PI 3.14` 标识符：仅可由数字、字母、下划线组成，且第一个字符必须为字母或下划线 `apple _apple a12581`
最终增加亿点点细节即可完成。

4) 其余在各部分内说明。

7.判断

1) **if**(判断条件，为真则执行，否则不执行)，常与 **else** 共同使用。

2) 多分支时，常以 `if() else if() .....`使用。

说明：`if` 后若希望有多个语句，则需要 `{}`，且 **if 和 else 遵循就近匹配原则**。`if` 的判断条件中可以为一个数字，为 0 表示假，非 0 表示真。也可以是一个逻辑条件，或合法语句等。`=`为赋值，`==`为判断相等，需要注意。

3) **switch** 语句：

```
int a;
scanf("%d",&a);
switch(a)
{
case 1:
;
break;
case 2:
;
break;
default:
;
break;
}
```

`switch()`和 `case` 后需要为整型常量

每个 `case` 语句后应加入 `break` 以跳出该 `switch` 语句，否则将继续执行之后的内容，建议在每个 `switch` 语句内加入 `default: {}`，以做默认选项，且在 `if` 和 `else` 中的内容外括 `{}`。

8.循环

1) **for** 循环使用较多，应重点掌握。**for**(初始化;判断条件;数值调整)，建议在 **for** 后的语句外括**{}**。

```
int i,a[10];for(i=0;i<10;i++){scanf("%d",&a[i]);}
```

且在写初始化条件和判断条件时建议采取**左闭右开**形式：**for(i=0;i<9;i++)**；该条件应该根据实际情况考虑是否使用。

2) **do{}while()**;循环：先执行 **do** 中内容，然后判断是否继续；

3) **while()**{}，判断是否进行。

说明：在写判断条件时，需要注意赋值和判断相等不同，同时注意 **break** 和 **continue** 的使用，以避免死循环。

9.函数（库函数（引头文件）+自定义函数（自己编写））

1) 参数：实际参数：语句中传输的；形式参数：函数后带的

```
int add(int a,int b);//形参; c=add(a,b);//实参;
```

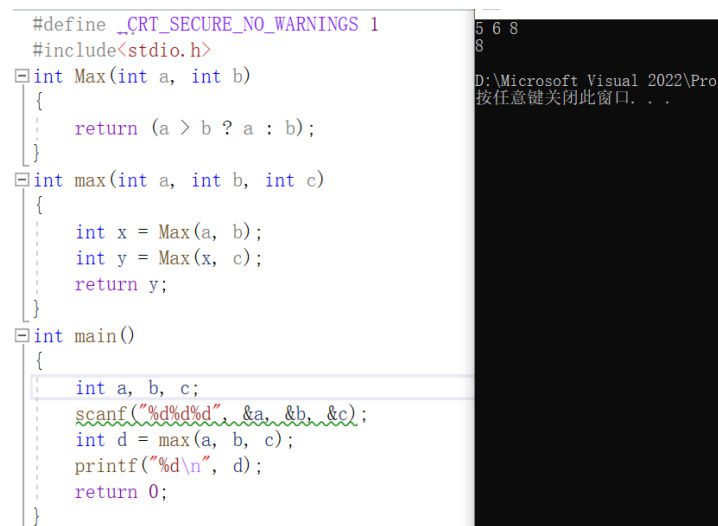
说明：实参必须有一个特定的值，该值可为一个式子（1+1），一个变量或常量，函数亦可。只有当实参传给形参时，形参才存在，同时可认为形参是实参的临时拷贝，对形参的改变不会直接影响到实参。当实参中包含一个数组，表面上看是个数组，但实际上是数组首个元素的地址，因此，在函数的形参中，表面上看是个数组，但实际是个指针。

```
int c(int a,int b[]);//实际为指针
```

```
int b,a[10];c=c(b,a);//实际为首元素地址
```

传值调用：形参改变不影响实参；传址调用：可以将函数和外部联系起来。

2) 函数嵌套：函数体里加函数。



```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int Max(int a, int b)
{
    return (a > b ? a : b);
}
int max(int a, int b, int c)
{
    int x = Max(a, b);
    int y = Max(x, c);
    return y;
}
int main()
{
    int a, b, c;
    scanf("%d%d%d", &a, &b, &c);
    int d = max(a, b, c);
    printf("%d\n", d);
    return 0;
}
```

3) 链式访问：一个函数的返回值带入另一个函数。

```
printf("%d",strlen("abc"));
```

4) 函数声明：若只在一个文件中时，当 `int main()` 在所调用的函数定义前，则需要提前声明。

```
int main(){int add(int,int);} int add(int a,int b){}
```

当函数定义在前，`int main()` 在后，则不需要声明。

```
int add(int a,int b){} int main(){return 0;}
```

说明：1.声明不关注函数实际是否存在。2.应先声明，后使用。3.函数声明一般放头文件（面向对象时常使用）。

5) 函数递归：可认为是函数自身调用自身，常见错误 `stack overflow`（栈溢出）：因为死循环导致栈空间耗尽。因此，在欲执行递归时，要明确其结束执行的条件，且让每次递归后都不断接近该条件。

```
#define _CRT_SECURE_NO_WARNINGS 1
#include <stdio.h>

void div(int a)
{
    int b;
    if (a > 9)
        div(a / 10);
    printf("%d ", a%10);
}

int main()
{
    int a = 1234;
    div(a);
    return 0;
}
```

6) 可变函数：函数的参数不定，可以控制

```
#define _CRT_SECURE_NO_WARNINGS 1
#include <stdio.h>
#include <stdarg.h>

double average(int num, ...)
{
    va_list valist;
    double sum = 0.0;
    int i;

    /* 为 num 个参数初始化 valist */
    va_start(valist, num);

    /* 访问所有赋给 valist 的参数 */
    for (i = 0; i < num; i++)
    {
        sum += va_arg(valist, int);
    }

    /* 清理为 valist 保留的内存 */
    va_end(valist);
    return sum / num;
}

int main()
{
    printf("Average of 2, 3, 4, 5 = %f\n", average(4, 2, 3, 4, 5));
    printf("Average of 5, 10, 15 = %f\n", average(3, 5, 10, 15));
    return 0;
}
```

10.数组（起始下标为 0，在内存中连续存放）

数组元素数量求法：

```
int a[]={1,2,3,4,5};int b=sizeof(a)/sizeof(a[0]);->b=5。
```

1) 一维数组创建：元素类型 数组名[元素个数]; `int a[10];`

数组类型：除去数组名的其他部分。`int a[10];`//该数组类型为 `int [10]`

2) 一维数组初始化：

未定义的初始化：根据后面的进行元素个数认定。`int a[]={1,2,3,4};`说明 `a` 有 4 个元素。

完全初始化：`int a[4]={1,2,3,4};`

不完全初始化：`int a[5]={1,2};` 后面的数值默认为 0。

说明：定义 `char a[4]`并不完全初始化时，后面的值也为 0，但需注意：

当如此初始化时：`char a[5]='a','b';`元素分别为：`a,b,0,0,0`。当如此初始化时：`char a[5]="ab";`元素分别为：`a,b,\0,0,0`。在初始化时，可以用 ASCII 码代表字符：

`char a[5]='a',98;`元素分别为：`a,b,0,0,0`。

3) 数组元素输入和输出：

对于非字符类型：需要 `for` 循环实现输入和输出。

`int i,a[10];for(i=0;i<10;i++)scanf("%d",&a[i]);`

对于字符类型：可以使用 `for`，但是注意每次输入输出一个元素，应该使用 `%c`。

或者可以：`char a[10];scanf("%s",a);printf("%s",a);`

4) 二维数组创建和初始化：元素类型 数组名[行数][列数]；先行后列

完全初始化（两种方法）以 `int a[3][4];`为例：共可存储 12 个元素

(1)`int i[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};`

(2)`int i[3][4]={{1,2,3,4},{5,6,7,8},{9,10,11,12}};`

不完全初始化（两种方法）以 `int a[3][4];`为例：共可存储 12 个元素

(1)`int i[3][4]={1,2,3,4,5,6,7,8};` 先排第一行，满了排第二行，其余默认初始化为 0。

(2)`int i[3][4]={{1,2,3},{5,6},{9}};`外 `{}` 中的每个 `{}` 表示一行，然后初始化里面的数字，其余为 0。->实际元素：`1,2,3,0`(第一行)`5,6,0,0`(第二行)`9,0,0,0`(第三行)

未定义的初始化：可以省略行，但不能省略列。`int a[][4]={{1,2},{3,4}};`

5) 数组的存储地址：数组名一般代表该数组首元素的地址。但存在例外：

1.**sizeof(数组名)**：此时该数组名表示整个数组。因此该语句表示：求这个数组占用的空间，单位字节。

2.**&数组名**：此时数组名表示整个数组。因此该语句表示，取出整个数组的地址。

(1).若希望打印出某数组的首元素地址时可以通过如下方法实现。

`printf("%p",arr);`或者 `printf("%p",&arr[0]);`

4) 类型重定义:

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    typedef struct stu
    {
        char name[20];
        char sex[5];
        int age;
    }stu;
    stu s1 = { "张三", "man", 20 };
    printf("%s %s %d", s1.name, s1.sex, s1.age);
    return 0;
}
```



使用此方法, `typedef struct` 结构体名{}修改名;之后即可用改后的名字定义变量。

5) 结构体嵌套: 结构体内可以嵌套另一个结构体, 在初始化和打印时需要注意。

```
{
    struct dmir
    {
        int p;
        char pp[5];
    };
    struct kula
    {
        int age;
        struct dmir ka;
        char name[20];
    };
    struct kula a = { 20, {6, "652"}, "kuladmir" };
    printf("%d\n%d\n", a.age, a.ka.p);
    return 0;
}
```



6) 结构体内存对齐规则: 第一个成员在与结构体变量偏移量为 0 的地址处, 其他成员变量要是对齐到对齐数整数倍的地址处, 对齐数=编译器的对齐数与该成员大小的较小值, 结构体大小为最大对齐数的整数倍, 如果结构体内嵌套结构体, 则嵌套的结构体对齐到自己的最大对齐数的整数倍处。=>适当安排成员变量的定义顺序, 以节省空间。

修改默认对齐数: `#pragma pack(设置数)` 之后输入: `#pragma pack()`表示取消设置

`offsetof` 用于计算结构体某变量的偏移量。

用法: `offsetof(struct 结构体名, 成员名);`需要`#include<stddef.h>`

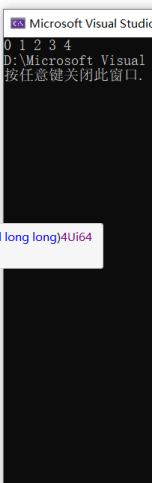
7) 位段: 成员为 `int`、`unsigned int` 等, 定义时需要在变量后加:和一个数字 (<=32)。

定义方式: `struct S{int a:5; int b:10; int c:20;};`

说明：后面的数字表示这个变量占有的 **bit** 位，但是也有其存储规则：位段是按照 **int** 或 **char** 方式开辟空间，即 4 字节或 1 字节，当一个整型字节剩余的空间不够给一个变量时，将重新开辟一个整形字节空间。

8) 柔性数组：结构体中最后一个成员为数组时，可以为未知大小。在计算此结构体大小时，不计算柔性数组的大小，和动态开辟内存函数共用。

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int i;
    struct S
    {
        int n;
        int a[]; //int a[0]也可
    };
    struct S* p = (struct S*) malloc(sizeof(struct S) + 5 * sizeof(int));
    if (p == NULL)
    {
        printf("开辟失败\n");
        return 0;
    }
    p->n = 100;
    for (i = 0; i < 5; i++)
    {
        p->a[i] = i;
        printf("%d ", p->a[i]);
    }
    free(p);
    p = NULL;
    return 0;
}
```



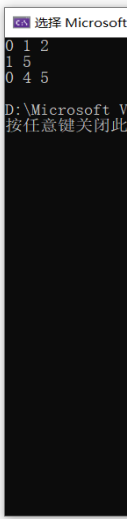
12.枚举（具有可读性、可维护性，便于调试，使用方便，有封装无干扰，更严谨）

1) 声明方式：**enum** 类型名{可能取值 1,可能取值 2,可能取值 3.....}; //最后一个可能后不加逗号。

2) 变量定义：**enum** 类型名 变量名 = 可能取值;

说明：枚举的可能取值中，每个都对应一个默认数字，从 0 开始递增，但是可以在声明中修改。

```
#include<stdio.h>
int main()
{
    enum colour
    {
        Red, //默认0
        Blue, //默认1
        Pink //默认2
    };
    enum colour2
    {
        Green = 1, //修改为1
        Yellow = 5 //修改为5
    };
    enum colour3
    {
        Brown, //默认0
        Orange = 4, //修改为4
        Gray //递增+1为5
    };
    printf("%d %d %d\n", Red, Blue, Pink);
    printf("%d %d\n", Green, Yellow);
    printf("%d %d %d\n", Brown, Orange, Gray);
    return 0;
}
```



13.联合体（成员共用一个空间）

1) 声明方式：union 类型名{成员变量;.....};

2) 空间大小计算：大小至少为最大成员大小，但是最大成员大小不是最大对齐数的整数倍，就要对齐到最大对齐数的整数倍。

14.指针（指针是个变量，用来存在地址的变量，32 位系统固定 4 字节。64 位固定 8 字节）

1) 指针类型：在 32 位系统下，所有指针都占 4 个字节，64 位下，占用 8 个字节。不同指针类型虽然均可以存放地址，**但是不同指针类型决定了解引用所能访问空间的大小**。int* 可以访问 4 个字节，char* 可以访问 1 个字节，double* 可以访问 8 个字节。**且决定了指针的步长**：指针+整数操作时，int* 型+1->走 4 个字节，char* 型+1->走 1 个字节。

void* 类型的指针不能解引用，且不能进行±整数的运算。

2) 野指针：指针指向的位置不可知的指针。未初始化和越界访问可能会导致此问题。

因此需要时刻注意对指针进行初始化，如果暂时没有初始化的想法，也需要将其设置为 NULL。

（指针为空指针时将不可使用），也可检测指针是否有效。

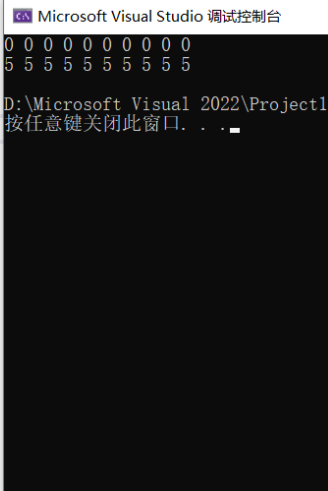
int* p;p=NULL;//此时指针为空指针，不可使用

3) 指针运算：

如果将数组名赋值给一个指针，表示将该数组首元素的地址赋值给此指针，见下图第五行。

(1)指针±数字：常用于数组内，可以通过加减实现对其他元素的控制，但需保证对同一内存空间使用。

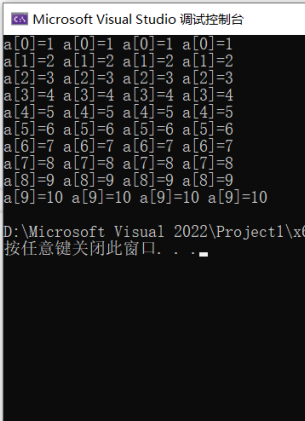
```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    int a[10] = {0};
    int* p = a;
    int i;
    for (i = 0; i < 10; i++)
        printf("%d ", a[i]);
    printf("\n");
    for (i = 0; i < 10; i++)
        *p++ = 5;
    for (i = 0; i < 10; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```



补充：当将数组的首元素地址传递给一个指针后，该数组和该指针即绑定在一起。

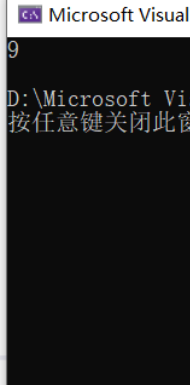
```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int* p = a;
    int i;
    for (i = 0; i < 10; i++)
    {
        //四种方法等价
        printf("a[%d]=%d ", i, a[i]);
        printf("a[%d]=%d ", i, p[i]);
        printf("a[%d]=%d ", i, *(p + i));
        printf("a[%d]=%d ", i, *(a + i));
        printf("\n");
    }

    return 0;
}
```



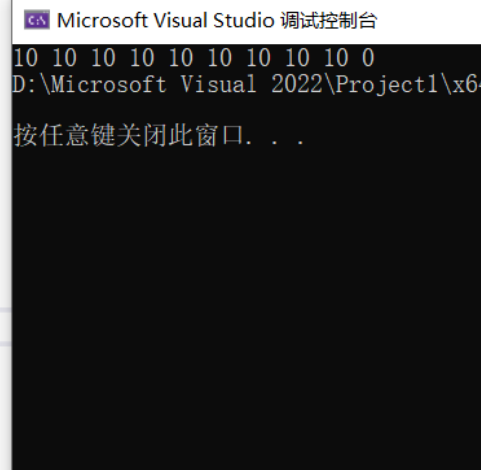
(2)指针-指针：常用于数组内使用，需使用同一空间内的大地址-小地址，得到中间的元素个数，但不能使用不同数组地址计算。

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    int a[10] = {0};
    char c[10] = { 1 };
    printf("%d\n", &a[9] - &a[0]);
    // 此句错误
    //printf("%d\n", &a[9] - &c[0]);
    return 0;
}
```



(3)指针比较大小。说明：标准仅支持数组指针和该数组后一位比较，不可和前一一位比较。

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    int a[10] = { 0 };
    int* p;
    for (p = &a[9]; p > &a[0];)
        *--p = 10;
    for (p = &a[0]; p <= &a[9]; p++)
        printf("%d ", *p);
    return 0;
}
```



4) 指针和数组关联：需通过指针和整数的加减实现，把握好其定义即可较好使用。

说明：int a=10;int* p=&a;此时 p 为 a 的地址，打印时需使用%p，而若想通过其简介访问 a，则需要解引用*p。

5) 二级指针：用以存放指针变量地址的指针变量。

定义：类型** 变量名;

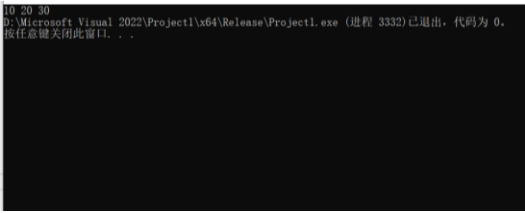
用法参考一级指针即可：解引用：****p**;间接访问：****p=10**;

```
#include<stdio.h>
int main()
{
    int a = 10;
    int* p = &a;
    int** pa = &p;
    printf("%p\n", &pa);
    printf("%d\n", **pa);
    return 0;
}
```



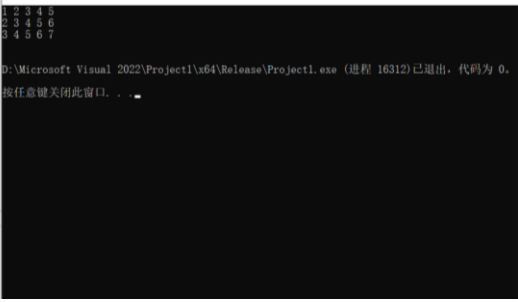
6) 指针数组：用以存放一组指针变量的数组，使用方法联合指针和数组即可。

```
#include<stdio.h>
int main()
{
    int a = 10;
    int b = 20;
    int c = 30;
    int e;
    int* d[3] = { &a, &b, &c };
    for (e = 0; e < 3; e++)
        printf("%d ", *d[e]);
    return 0;
}
```



常用指针数组实现类似二维数组的用法:

```
#include<stdio.h>
int main()
{
    int a[] = { 1, 2, 3, 4, 5 };
    int b[] = { 2, 3, 4, 5, 6 };
    int c[] = { 3, 4, 5, 6, 7 };
    int* p[] = { a, b, c };
    int i, j;
    for (i = 0; i < 3; i++)
    {
        for (j = 0; j < 5; j++)
            printf("%d ", *(p[i] + j));
        printf("\n");
    }
    printf("\n");
    return 0;
}
```

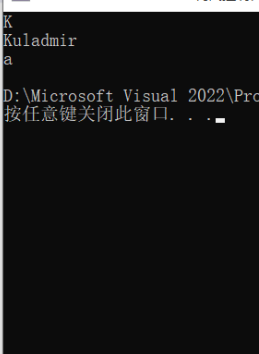


7) 字符指针：存放字符或字符串地址的变量

当指针指向字符地址时，像其他类型解引用即可。而当指向字符串首元素地址时，**不需要*解引用，使用%s即可输出字符串，而当解引用时，将输出第一个字符**。注意：不能用解引用的指针变量修改字符。

例如：**char a[]="kuladmir";char* p=a;*p='w';**//此句语法错误

```
#include<stdio.h>
int main()
{
    char d = 'a';
    char* b=&d;
    char a[] = "Kuladmir";
    char* p = a;
    printf("%c\n", *p);
    printf("%s\n", p);
    printf("%c\n", *b);
    return 0;
}
```



如果两个指针的初始化内容相同，则这两个指针指向相同地址。

char* a="abcdef";char* b="abcdef";//指针 a、b 都指向该字符串的首元素地址

如果将指针指向常量字符串，最好写成：**const char* a="abcdef";**

8) 数组指针：存放数组地址的指针

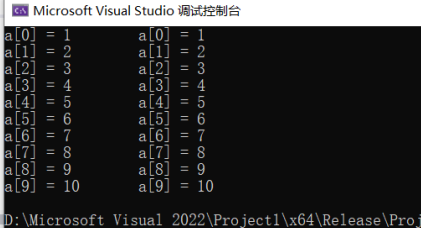
定义：int a[10];int (*p)[10];//表示指针 p 指向一个有 10 个元素的整型数组

//例题：int (*a[10])[5];

表示 a 是一个数组，有 10 个元素，每个元素是个指针，每个指针指向一个有 5 个元素的整型数组。

类型：除去变量名的其他部分。如上例：int (*)[5] 为 a[10] 的类型。

```
#include<stdio.h>
int main()
{
    int a[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    int (*p)[10]=&a;//数组指针的定义
    char* b[5];
    int i;
    char* (*pa)[5]=&b;//数组指针的定义，此类似于二次指针
    //详解：*pa说明pa是个指针，[5]说明pa指向一个数组，char* 说明pa指向数组的元素类型
    //数组指针的简单用法
    for (i = 0; i < 10; i++)
    {
        printf("a[%d] = %d ", i, (*p)[i]);
        printf("a[%d] = %d\n", i, (*p + i));
    }
    return 0;
}
```

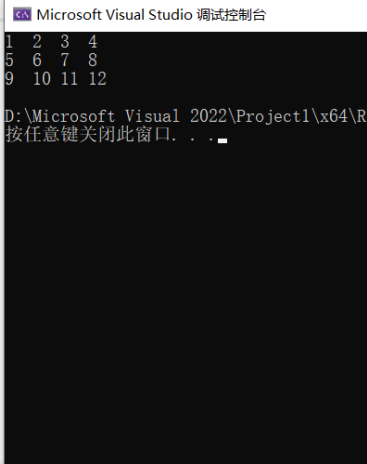


数组指针常在二维数组中使用：

补充：二维数组在函数中传参时，传过去的是第一行的地址。

示例：void a(int (*p)[5]){ int main(){ int b[3][5]={0};a(b);return 0; }

```
#include<stdio.h>
void print(int (*p)[4], int x, int y)
//当然可以写成 void print(int a[3][4], int x, int y)
{
    int i, j;
    for (i = 0; i < x; i++)
    {
        for (j = 0; j < y; j++)
            printf("%-2d ", *((p+i) + j));
        printf("\n");
    }
}
int main()
{
    int a[3][4] = { {1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12} };
    int i;
    print(a, 3, 4);
    return 0;
}
```

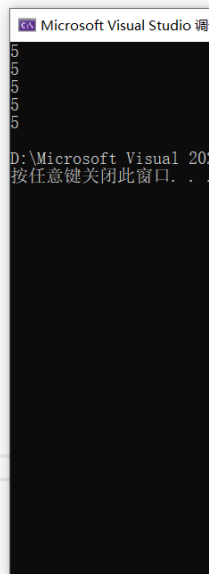


9) 指针传参 (2 种情况)

```

#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
void arr(int* p)//一级指针时的传参如下
{
    printf("%d\n", *p);
}
void arr2(int** p)//二级指针时的传参如下
{
    printf("%d\n", **p);
}
int main()
{
    int a = 5;
    int* p = &a;
    arr(p);//一级 - 一个指针变量
    arr(&a);//一级 - 一个地址
    int* b[10] = { &a };
    int** pp = &p;
    arr2(&p);//二级 - 一个一级指针的地址
    arr2(pp);//二级 - 一个二级指针
    arr2(b);//二级 - 一个指针数组的首元素地址
    return 0;
}

```



10) 函数指针：指向函数的指针 函数名和&函数名 -> 该函数的地址

定义：int add(int x,int y){}; int (*p)(int,int)=&add; //根据实际定义

类型：除去变量名的其他部分，本例：int (*)(int , int)

函数指针类型重定义：typedef void(* pfun_t)(int)//斜体部分为重定义后的关键字，表示将类型：void (*)(int)改名为 pfun_t

//例题：(*(void(*)())0)0

先隔开(* (void(*)()) 0)0：加粗部分表示强制类型转换，将 0 强制转换为函数指针类型，之后调用地址为 0 的函数。

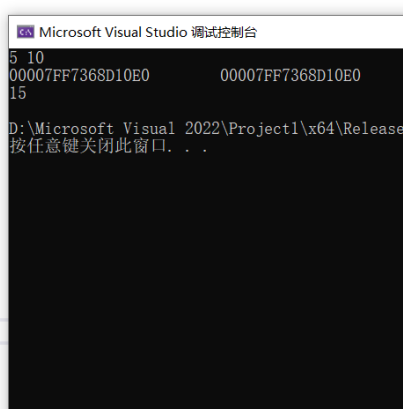
//例题：void(* signal(int,void (*)(int)))(int)

signal 是个函数，其两个参数分别为 int 类型和函数指针类型（参数为 int，返回值为 void），signal 函数的返回类型为函数指针类型（参数为 int，返回值为 void）。

```

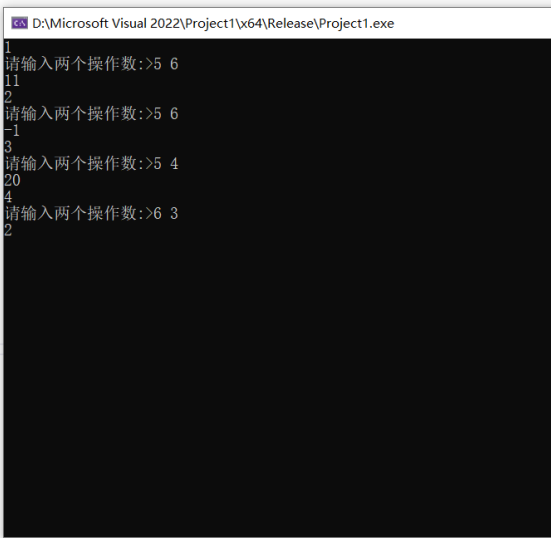
#include<stdio.h>
int arr(int x, int y)
{
    return x + y;
}
int main()
{
    int a, b;
    scanf("%d%d", &a, &b);
    int c = arr(a, b);
    printf("%p ", &arr);//打印函数地址
    printf("%p\n", arr);//打印函数地址
    int (*p)(int, int) = arr;//函数指针
    printf("%d\n", (*p)(a, b));//简介调用函数
    return 0;
}

```



当使用函数指针指向一个函数时（即保存了一个函数的地址），在调用时可以不用解引用符号。例如上图中第 14 行，可直接写成：printf("%d\n",p(a,b)); 可以让某函数的参数为一个函数指针，用以接收某函数的地址，以实现更多功能。

```
#include<stdio.h>
void maths(int (*p)(int , int ))
{
    int a, b;
    printf("请输入两个操作数:>");
    scanf("%d%d",&a,&b);
    printf("%d\n", p(a, b));
}
int add(int x, int y)
{
    return x + y;
}
int sub(int x, int y)
{
    return x - y;
}
int mul(int x, int y)
{
    return x * y;
}
int div(int x, int y)
{
    return x / y;
}
int main()
{
    int input;
    do
    {
        scanf("%d",&input);
        switch (input)
        {
            case 1:
                maths(add);
                break;
            case 2:
                maths(sub);
                break;
            case 3:
                maths(mul);
                break;
            case 4:
                maths(div);
                break;
            case 0:
                break;
            default:
                break;
        }
    } while (input);
    return 0;
}
```

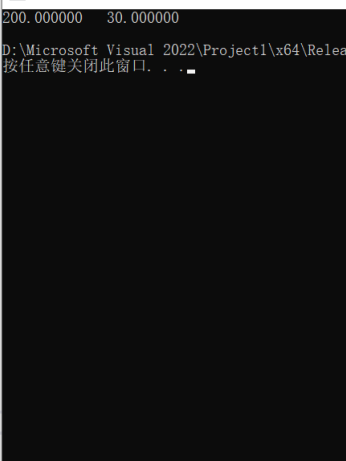


11) 函数指针数组：一个存储函数地址的数组。需要存储的函数有相同的返回类型和参数类型。

定义：int (*p[4])(int, int)={}; //根据实际定义

类型：除去数组名的其他部分，本例：int (*[4])(int, int)

```
#include<stdio.h>
double a(double x, double y)
{
    return x + y;
}
double b(double x, double y)
{
    return x * y;
}
int main()
{
    double (*p[2])(double, double) = {a, b};
    double a = 10, b = 20;
    double c = p[1](a, b);
    double d = p[0](a, b);
    printf("%lf %lf\n", c, d);
    return 0;
}
```



用途：转移表，用以实现计算器功能时减少 case 语句长度。（如下图第 28 行）



15. 数据存储

1) 类型分类：内置类型（**int char short** 等），自定义类型。

整型：**char short int long**（此处包括 **unsigned** 和 **signed**），无符号数的范围比有符号数的范围大。

浮点型：**float double**。

构造类型：**数组类型 struct enum union**。说明：**数组的类型为其除去名字的其他部分**。例如：**int a[10];**//**int [10]**为该数组的类型。

指针类型：**int* char*** 等。

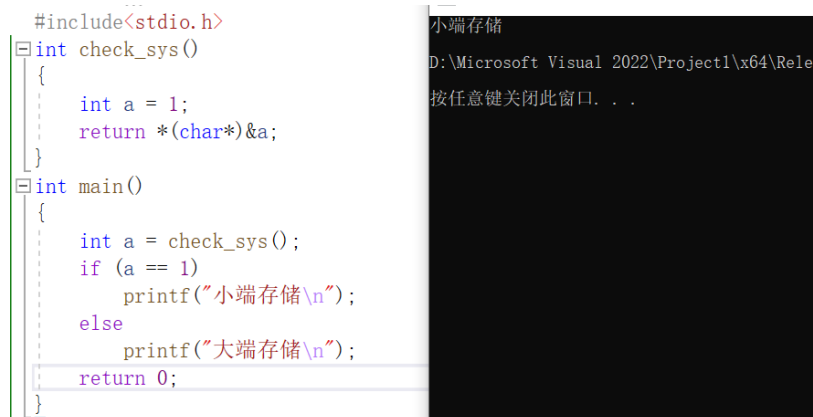
空类型：**void** 常用于函数返回类型。

2) 整型的存储方式

原码、反码、补码：对于整型有符号数，最高位表示正负，其他位是数值位，1为负数，0为整数，对于正数，三码合一，而计算机存储补码。原码通过十进制数字直接转换而成，反码通过对原码除符号位按位取反得到，补码通过反码+1得到。

3) 大小端

大端存储模式：数据低位存在内存高地址处；**小端存储模式**：数据低位存在内存低地址处，可以通过代码方式得到计算机的存储方式。



4) 浮点型数据的存储方式

IEEE 754 规定：一个浮点数可以被写成 $(-1)^S * M * 2^E$ ，通过存储 S、M、E 即可存储该数字。对于 **float** 类型：1 bit 位存储 S，8 bit 位存储 E，23 bit 位存储 M。对于 **double** 类型：1 bit 位存储 S，11 bit 位存储 E，52 bit 位存储 M。同时也规定：保存 M 时默认其第一位是 1，因而舍去，只保存后面的部分，读取时再加上去。

特别规定：M 的范围[1,2)；如果该数字为正，则 S=0，负则 S=1；存储 E 时，E 是个无符号整数，但是在存入内存时，E 的真实值需要加上一个中间数，8 位时是 127，11 位是 1023。

对于十进制(float)9.0，二进制表示形式为：1001.0，可化为 $(-1)^0 * 1.001 * 2^3$ 。此时 S=0，E=3，M=1.001，但是实际内存存储的是：E 为 3+127=130，M 为 001，S 为 0。对于十进制(float)5.5，二进制表示形式为：101.1，可化为： $(-1)^0 * 1.011 * 2^2$ 。此时内存存储形式为：

0 10000001 011000000000000000000000 ->
01000000101100000000000000000000

在取出时,有如下情况：如果 E 不全为 0 或 1，则在取出时，给 M 前补 1，E 的存储值-127 得到真实值，之后还原成数值；如果 E 全为 0，则在取出时，不给 M 前补 1，E 的值为 -126 或 -1022；如果 E 全为 1，则表示正（负）无穷大（小）。

对于：01000000101100000000000000000000 -> 0 10000001
01100000000000000000000000000000，发现 E 的二进制序列不全为 0 或 1，则还原时：
E(10000001->129)-127 = 2，M = 1.011，最终还原为： $(-1)^0 * 1.011 * 2^2$
-> 5.5。

对于：00000000001100000000000000000000 -> 0 00000000
01100000000000000000000000000000，发现 E 的二进制序列全 0，则还原时：E
(00000000) = -126，M = 0.011，最终还原为：
 $(-1)^0 * 0.011 * 2^{-126}$ 。

16.内存动态分布（堆区的任务）

1) 回忆：内存空间主要划分为 3 个区域：栈区、堆区、静态区。

栈区：存放局部变量和函数形式参数。

静态区：存放全局变量、静态变量（static）。

2) 动态内存分配（需要#include<stdlib.h>）：

malloc 函数：用以开辟内存块。函数形式：**void* malloc (size_t size);**

用法：指针类型 p =(指针类型) malloc(欲开辟的空间大小（字节）);

free 函数：用以释放空间。函数形式：**void free (void* ptr);**

用法：free(p);//指向开辟空间的指针为参数。

calloc 函数：用以开辟一个空间，并且初始化为 0。

函数形式：**void* calloc (size_t num, size_t size);**

用法：(指针类型)p=(指针类型)calloc(元素数量，每个元素长度);

realloc 函数：用以调整开辟空间的大小。

函数形式：**void* realloc (void* ptr, size_t size);**

用法：(指针类型)p=(指针类型)realloc(已开辟空间的指针，新的空间大小（字节）);

realloc 函数可以实现 malloc 函数的作用：

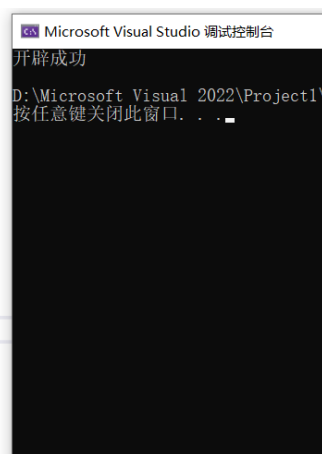
int* p=(int*)realloc(NULL,40);//等价于 int* p=(int*)malloc(40);

说明：使用 **malloc(calloc)**函数时，需要对返回值进行检查：开辟失败返回空指针，可使用 **strerror(errno)**检测错误，开辟成功返回空间地址；

malloc(calloc)和 **free** 需要成对使用。释放空间后，应将该指针赋值为 **NULL**。

使用 **realloc** 函数时，存在两种情况：申请增加空间后，如果开辟的空间后有足够的空间，则进行追加，返回原地址，如果不够，则会重新开辟一个新的符合要求的空间，将之前的数据拷贝放入新空间，返回新地址，因此建议用新的指针接收该地址。不能对开辟的内存空间进行越界访问，不能对非动态开辟的空间进行 **free** 释放，如果指向开辟空间的指针改变了（不指向首位），则不能使用 **free** 释放。

```
#include<stdio.h>
#include<string.h>
#include<errno.h>
#include<stdlib.h>
int main()
{
    int* p = (int*)malloc(40);
    if (p == NULL)
        printf("%s\n", strerror(errno));
    else
        printf("开辟成功\n");
    int* p2 = (int*)calloc(5, 4);
    int* p3 = (int*)realloc(p, 40);
    free(p3);
    return 0;
}
```



17.文件操作

1) 文件类型与概念：文本文件（以 ASCII 形式存储，需要在存储前转换的文件）、二进制文件（数据以二进制形式存储，不转换的输出到外存）。文件缓冲区：用于暂时储存数据的空间，当空间满后再输出和输入。文件指针：打开文件时，会自动创建一个该文件的相关信息指针 **FILE***。标准输入输出流：

stdin-标准输入流，FILE*类型，指用键盘输入内容，stdout-标准输出流，FILE*类型，指用屏幕输出内容。

2) 文件的相关操作：

fopen 函数：函数形式：**FILE * fopen (const char * filename, const char * mode);**用法：fopen(“文件名”,“模式选择”); 打开成功返回指针，失败返回空指针。

说明：绝对路径写法：在文件名处加上具体的路径；相对路径：../表示上一路径，使用此方法打开文件。

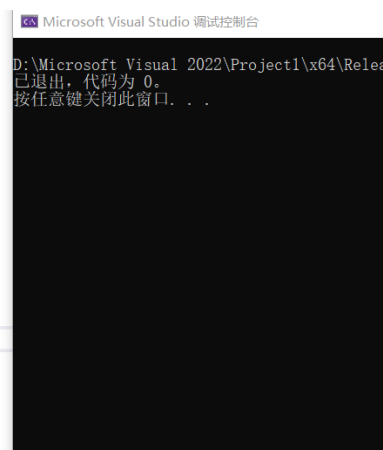
模式：“r”：打开文本文件，不存在时会报错。“w”：创建文本文件并准备写入，如果文本文件不存在则创建，如果存在则弃去原有内容。“a”：打开文本文件并在文件末尾写入，文件不存在时则创建。“rb”：打开二进制文件，不存在时会报错。“wb”：创建二进制文件并准备写入，如果文本文件不存在则创建，如果存在则弃去原有内容。“ab”：打开文本文件并再文件末尾写入，文件不存在时则创建。“r+”：打开文本文件并写入，不存在时会报错。“w+”：创建文本文件并准备写入，如果文本文件不存在则创建，如果存在则弃去原有内容。“a+”：打开文本文件并在文件末尾写入，文件不存在时则创建。可以混合+、b、rwa模式使用。

fclose 函数：函数形式：**int fclose (FILE * stream);**用法：fclose(文件指针);
说明：在使用 fopen 函数后，要像 malloc 等函数一样，判断指针是否为空，使用 fclose 函数后，要将指针设置为 NULL。

fputc 函数：用于将一个字符写入文件，写入源可以为文件指针，此时将写入文件，也可以是 stdout，如此将把此字符输出，但不写入文件。函数形式：**int fputc (int character, FILE * stream);**用法：fputc(‘字符’,文件指针);

fgetc 函数：用于打印文件中的内容，读取源可以为文件指针，此时需要一个 int 变量接收，也可以是 stdin，此时需要从键盘输入一个字符。函数形式：**int fgetc (FILE * stream);**用法：fgetc(文件指针);

```
#include<stdio.h>
#include<string.h>
#include<errno.h>
int main()
{
    FILE*p = fopen("test.txt", "w");
    if (p == NULL)
    {
        printf("%s\n", strerror(errno));
        return 0;
    }
    fputc(97, p);
    fputc(98, p);
    fclose(p);
    p = NULL;
    return 0;
}
```

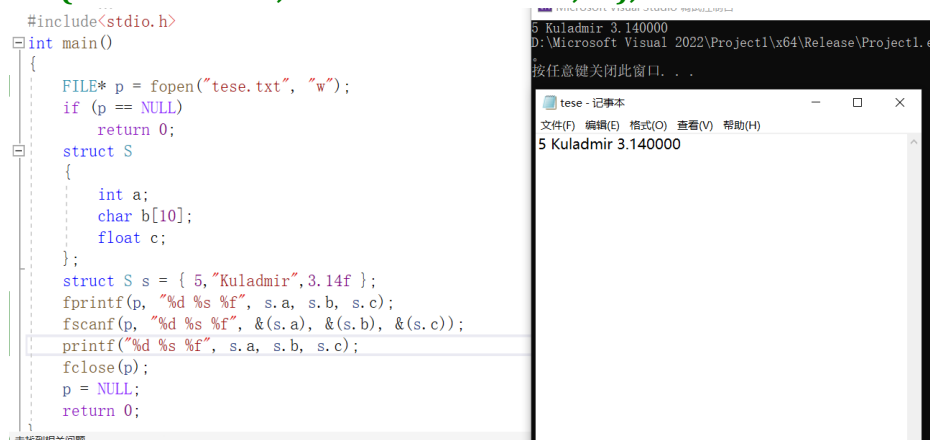


fgets 函数：用于把文件中的内容存储到字符串，并可设定长度，文件中有换行符时会终止读取，**不会自己增加换行符**，可以把文件指针写成 stdin，即从键盘上输入字符串。函数形式：**char * fgets (char * str, int num, FILE * stream);**用法：fgets(字符串，设置长度，文件指针);

fputs 函数：用于写入一个字符串到文件。函数形式：**int fputs (const char * str, FILE * stream);**用法：fputs(字符串，文件指针);

fprintf 函数：用于格式化写入文件。可以使用 stdout。函数形式：**int fprintf (FILE * stream, const char * format, ...);**

fscanf 函数：用于将文件中内容读取放到某位置，可以使用 stdin。函数形式：**int fscanf (FILE * stream, const char * format, ...);**



```
#include<stdio.h>
int main()
{
    FILE* p = fopen("tese.txt", "w");
    if (p == NULL)
        return 0;
    struct S
    {
        int a;
        char b[10];
        float c;
    };
    struct S s = { 5, "Kuladmir", 3.14f };
    fprintf(p, "%d %s %f", s.a, s.b, s.c);
    fscanf(p, "%d %s %f", &(s.a), &(s.b), &(s.c));
    printf("%d %s %f", s.a, s.b, s.c);
    fclose(p);
    p = NULL;
    return 0;
}
```

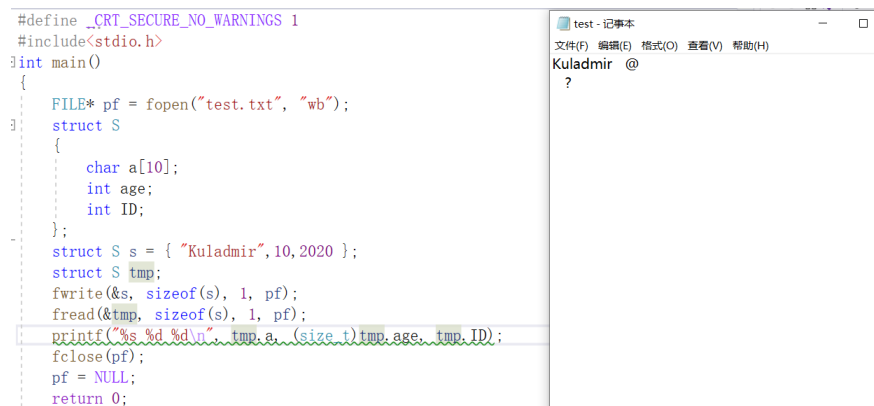
fwrite 函数：把二进制数据写入文件,需要以二进制形式打开文件。函数形式：**size_t fwrite (void * ptr, size_t size, size_t count, FILE * stream);**用法：

fwrite(待读取元素地址,待写入元素大小,待写入元素个数,文件指针);

fread 函数：读取二进制文件打开，需要以二进制形式打开文件。函数形式：

size_t fread (const void * ptr, size_t size, size_t count, FILE * stream);用法：

fread(待写入地址,待读取元素大小,待读取元素个数,文件指针);



```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    FILE* pf = fopen("test.txt", "wb");
    struct S
    {
        char a[10];
        int age;
        int ID;
    };
    struct S s = { "Kuladmir", 10, 2020 };
    struct S tmp;
    fwrite(&s, sizeof(s), 1, pf);
    fread(&tmp, sizeof(s), 1, pf);
    printf("%s %d %d\n", tmp.a, (size_t)tmp.age, tmp.ID);
    fclose(pf);
    pf = NULL;
    return 0;
}
```

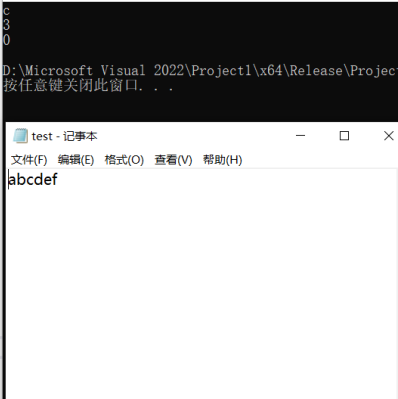
fseek 函数：用于设置文件指针指向位置。函数形式：**int fseek (FILE * stream, long int offset, int origin);**用法：fseek(文件指针,设置偏移量,偏移参考位);

偏移参考位：**SEEK_SET** 表示文件开头，**SEEK_CUR** 表示文件指针当前位置，**SEEK_END** 表示文件结束。

ftell 函数：用于得到文件指针偏移量。函数形式：**long int ftell (FILE* stream);**用法：int a=ftell(文件指针);

rewind 函数：用于重置文件指针偏移量。函数形式：**void rewind (FILE * stream);**用法：rewind(文件指针);

```
#include<stdio.h>
int main()
{
    FILE* p = fopen("test.txt", "r");
    if (p == NULL)
        return 0;
    fseek(p, 2, SEEK_SET);
    int a = fgetc(p);
    printf("%c\n", a);
    int b = ftell(p);
    printf("%d\n", b);
    rewind(p);
    int c = ftell(p);
    printf("%d\n", c);
    fclose(p);
    p = NULL;
    return 0;
}
```



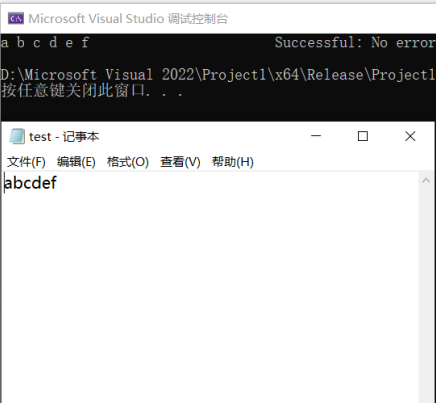
3) 文件结束判断 (EOF – end of file - 文件结束表示)

feof 函数：不能用此函数返回值判断文件是否结束，而用于文件读取结束后，判断读取是失败结束还是遇到文件尾结束，如果是遇到文件结束标志而结束，返回非零值，否则返回零。函数形式：**int feof (FILE * stream);**用法：feof(文件指针);

perror 函数：会把里面输入的内容输出，如果出错会在前面内容后输出错误原因。

ferror 函数：用以检测是否有错误，如果是返回非零值，否则返回零。函数形式：**int ferror (FILE * stream);**用法：if(ferror(文件指针))

```
#include<stdio.h>
int main()
{
    FILE* p = fopen("test.txt", "r");
    int c;
    if (p == NULL)
        perror("The problem is");
    while ((c = fgetc(p)) != EOF)
        printf("%c ", c);
    if (ferror(p))
        printf("Problem(s)\n");
    else if (feof(p))
        perror("Successful");
    fclose(p);
    p = NULL;
    return 0;
}
```



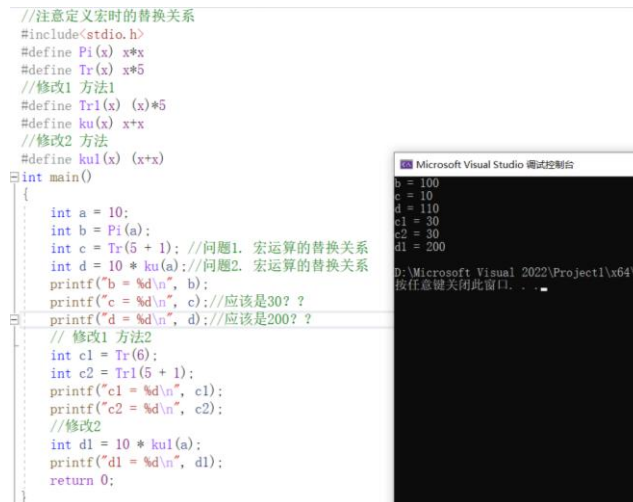
18.宏的定义和使用

1) #define 定义宏: #define 标识符(变量列表) 表达式

说明: 宏是完成替换的, 而不是传参, 需要注意加括号, 但常量字符串中的 **#define** 定义的变量, 将不会被替换; 定义宏时, 要把(变量列表)和标识符紧靠一起。宏参数和**#define** 的定义中可以存在其他**#define** 定义得到变量, 但是宏无法递归。注意尽量不要让宏的参数有副作用(类似于 **a++**)。

函数和宏: 宏相比于函数, 对参数类型要求较少, 而且没有函数调用和返回的时间消耗, 效率相对较高, 宏可以实现一些函数无法完成的任务。但是宏不能调试, 如果宏过长, 则需要替换多行语句, 造成语句长度过长。函数和宏互补使用将有很好的效果。

#undef 标识符 可以清除一个已定义的宏或标识符。



```
//注意定义宏时的替换关系
#include<stdio.h>
#define Pi(x) x*x
#define Tr(x) x*5
//修改1 方法1
#define Tr1(x) (x)*5
#define ku(x) x+x
//修改2 方法
#define kul(x) (x+x)
int main()
{
    int a = 10;
    int b = Pi(a);
    int c = Tr(5 + 1); //问题1. 宏运算的替换关系
    int d = 10 * ku(a); //问题2. 宏运算的替换关系
    printf("b = %d\n", b);
    printf("c = %d\n", c); //应该是30? ?
    printf("d = %d\n", d); //应该是200? ?
    //修改1 方法2
    int c1 = Tr(6);
    int c2 = Tr1(5 + 1);
    printf("c1 = %d\n", c1);
    printf("c2 = %d\n", c2);
    //修改2
    int d1 = 10 * kul(a);
    printf("d1 = %d\n", d1);
    return 0;
}
```

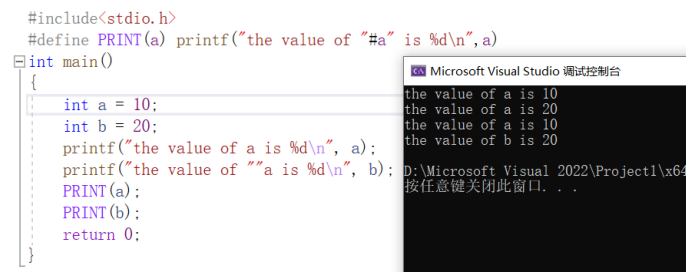
Microsoft Visual Studio 调试控制台

```
b = 100
c = 10
d = 110
c1 = 30
c2 = 30
d1 = 200
D:\Microsoft Visual 2022\Project1\src
按任意键关闭此窗口. . .
```

2) 宏的应用

宏可以实现字符串中的替换:

printf 函数中要打印的内容, 可以拆成多个""组成的字符串, 而需要调整变量打印时, 需要使用此方法, 并且使用 **#变量** 的方法。



```
#include<stdio.h>
#define PRINT(a) printf("the value of \"#a\" is %d\n",a)
int main()
{
    int a = 10;
    int b = 20;
    printf("the value of a is %d\n", a);
    printf("the value of \"a\" is %d\n", b);
    PRINT(a);
    PRINT(b);
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
the value of a is 10
the value of a is 20
the value of a is 10
the value of b is 20
D:\Microsoft Visual 2022\Project1\src
按任意键关闭此窗口. . .
```

变量**##**变量: **##**会将两边的变量组合成一个新的变量, 但此变量必须合法。

#define Ca(x,y) x##y int Cla15 = 10; printf("%d\n",Ca(Cla,15));

执行此代码将返回 10。

19.预处理

1) 翻译环境

每个源文件会经过编译器生成对应的目标文件，目标文件最后汇总到链接器生成可执行程序。由编译器处理的过程叫编译，链接器处理叫链接。

预编译阶段：对头文件的包含、清除注释（把注释改为空格）、**#define** 的替换（把定义的变量修改为定义值），.c 文件变成 .i 文件。**编译阶段：**把 .c 文件编译成 .s 文件（汇编代码），进行语言分析、词法分析、语义分析、符号汇总。**汇编阶段：**把 .s 文件转换成二进制代码，即 .o 文件，并产生一个符号表。**链接：**.o 文件存在一定格式，即 elf。合并段表：将对应段上的数据合并。并把符号表中的内容合并替换。

2) 运行环境

程序执行时，一般通过操作系统完成载入内存，然后打开。程序执行时开始调用 main 函数，之后将使用运行时堆栈，存储函数的局部变量和地址。

3) 预编译阶段

预定义符号

__FILE__ 表示此文件的路径，**__LINE__** 表示此行代码所在的行号，**__DATE__** 表示获取日期，**__TIME__** 表示获取时间，**__FUNCTION__** 表示此代码所在的函数。

预处理指令

#define 定义标识符和宏，标识符的值可以为数字、字符、关键字。在预编译阶段时，系统会将 **#define** 定义的符号替换，替换后的内容将被放在原位，对于宏，参数名将被其值替换。

4) 预处理指令

条件编译指令 1: #ifdef 标识符 内容 #endif

判断是否已存在 **#define** 定义的标识符，如果已被定义则编译内容中的语句，否则不编译。

条件编译指令 2: #if 常量表达式 内容 #endif

判断常量表达式是否为真，如果为真则编译内容中语句，否则不编译。

多分支条件编译指令: #if 常量表达式 内容 #elif 常量表达式 内容 #else 内容 #endif

可以实现多分支判断是否编译语句。

条件编译指令 3: #if defined(标识符) 内容 #endif

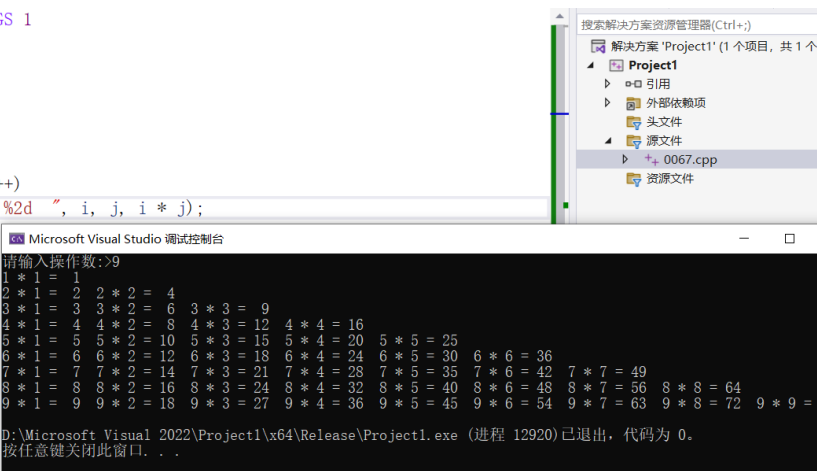
判断是否已存在 **#define** 定义的标识符，如果已被定义则编译内容中的语句，否则不编译。

文件包含: #include<文件名> 编译时直接去标准区查验，**#include"文件名"** 编译时去源路径查验，没有再去标准区查验，都没有则报错。

EX.经典例题（加深为重点）

1.编写一个程序，打印九九乘法表；

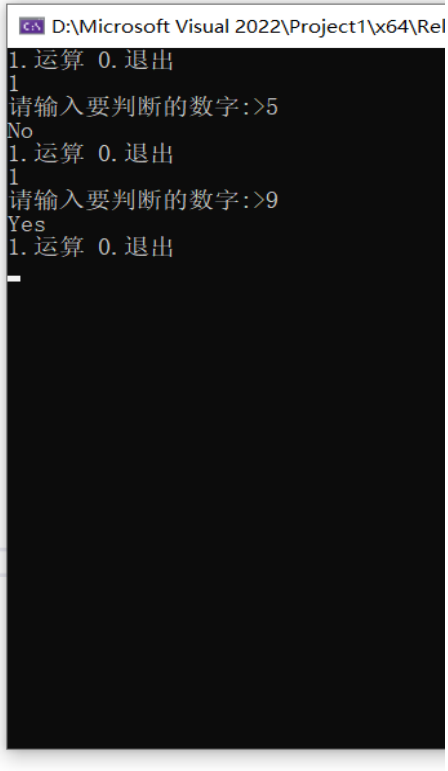
```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
void math_pri(int a)
{
    int i, j;
    for (i = 1; i <= a; i++)
    {
        for (j = 1; j <= i; j++)
            printf("%d * %d = %2d ", i, j, i * j);
        printf("\n");
    }
}
int main()
{
    int a;
    printf("请输入操作数:>");
    scanf("%d", &a);
    math_pri(a);
    return 0;
}
```



2.编写一个程序，判断一个数是否为素数；（试除法）

思路：从 2 到 $\sqrt{\text{number}}$ 进行取余测试，如果在范围内被整除，则不是质数，否则是。

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
#include<math.h>
int main()
{
    int a, b;
    do
    {
        printf("1. 运算 0. 退出\n");
        scanf("%d", &b);
        if (b == 0)
        {
            printf("退出");
            break;
        }
        printf("请输入要判断的数字:>");
        scanf("%d", &a);
        for (b = 2; b <= sqrt(a); b++)
        {
            if (a % b == 0)
            {
                printf("Yes\n");
                break;
            }
        }
        if (b > sqrt(a))
            printf("No\n");
    } while (b);
    return 0;
}
```



2+.在给定范围内查找素数；

```

//找素数
#define N 1
#define M 100
#include<stdio.h>
#include<math.h>
void prime(int x, int y)
{
    int i, number, count=0;
    for (i = x; i <= M; i++)
    {
        for (number = 2; number <= sqrt(i); number++)
        {
            if (i % number == 0)
                break;
        }
        if (number > sqrt(i))
        {
            printf("%d", i);
            count++;
        }
    }
    printf("\n");
    printf("By searching, we found that there is(are) %d primes from %d to %d\n", count, x, y);
}

int main()
{
    prime(N, M);
    return 0;
}

```

```

1 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
By searching, we found that there is(are) 26 primes from 1 to 100

C:\Users\11750\Desktop\事务文件夹\C\test\x64\Debug\test.exe (进程 29516)已
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动
按任意键关闭此窗口。...

```

3.编写一个程序，在一个有顺序的数列里，找一个数；（二分法）

思路：在循环外，先设定两端索引值；循环内，通过在二分范围内查找，逐渐减少二分范围，最终找到，最坏的结果是两端索引值相等（即在最中间），否则数列中没有这个数。

```

#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    int a[10];
    int i, j, left, mid, right, ser;
    printf("请输入这组从小到大数字:>");
    for (i = 0; i < 10; i++)
        scanf("%d", &a[i]);
    printf("请输入要查找的数字:>");
    scanf("%d", &ser);
    left = 0;
    right = 9;
    do
    {
        mid = (left + right) / 2;
        if (a[mid] > ser)
            left = mid + 1;
        else if (a[mid] < ser)
            right = mid - 1;
        else
        {
            printf("找到了，是%d个数字\n", mid + 1);
            break;
        }
    } while (left <= right);
    if (left > right)
        printf("未找到\n");
    return 0;
}

```

```

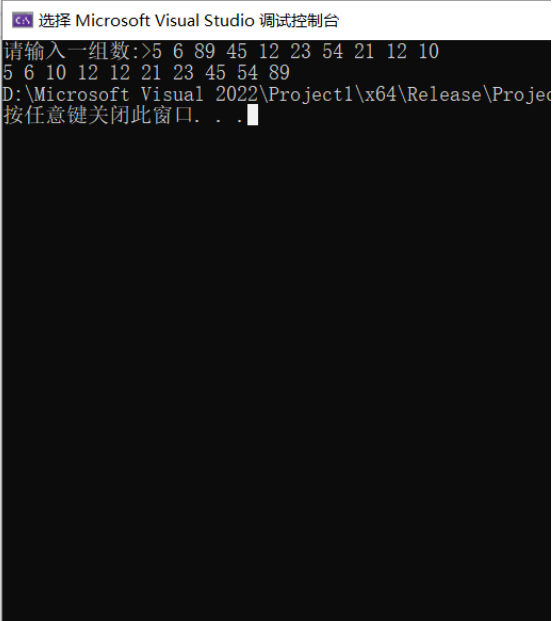
Microsoft Visual Studio 调试控制台
请输入这组从小到大数字:>1 2 3 4 5 6 8 9 10 12
请输入要查找的数字:>5
找到了，是5个数字

D:\Microsoft Visual 2022\Project1\x64\Release\Pr
按任意键关闭此窗口。...

```

4.编写一个程序，将 10 个数字从大到小排列并输出；（冒泡排序）

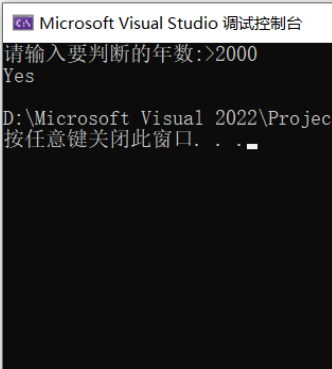
```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    int a[10];
    int i, j, tmp;
    printf("请输入一组数:>");
    for (i = 0; i < 10; i++)
        scanf("%d", &a[i]);
    for (i = 0; i < 9; i++)
    {
        for (j = 0; j < 9 - i; j++)
        {
            if (a[j] > a[j + 1])
            {
                tmp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = tmp;
            }
        }
    }
    for (i = 0; i < 10; i++)
        printf("%d ", a[i]);
    return 0;
}
```



5.编写一个程序，判断一年是否为闰年；

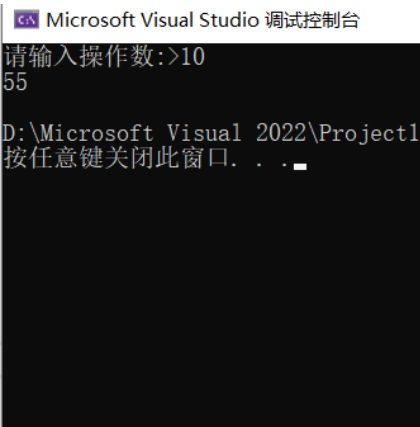
思路：年份能被 4 且不能被 100 整除，或者能被 400 整除的。

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    int a;
    printf("请输入要判断的年数:>");
    scanf("%d", &a);
    if ((a % 4 == 0 && a % 100 != 0) || a % 400 == 0)
        printf("Yes\n");
    else
        printf("No\n");
    return 0;
}
```



6.编写一个程序，计算 1+2+.....+n;

```
#include<stdio.h>
int main()
{
    int a, b, i;
    printf("请输入操作数:>");
    scanf("%d", &a);
    for (i = 0, b = 0; i <= a; i++)
        b = b + i;
    printf("%d\n", b);
    return 0;
}
```



7.编写一个程序，找出两个数的最大公约数和最小公倍数；（辗转相除法）


```

#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    int a, b, c;
    printf("请输入要计算的两个数:>");
    scanf("%d%d", &a, &b);
    do
    {
        c = a % b;
        a = b;
        b = c;
    } while (b);
    printf("%d\n", a);
    return 0;
}

```

Microsoft Visual Studio 调试控制台

请输入要计算的两个数:>24 18
6

D:\Microsoft Visual 2022\Project1\x64\
按任意键关闭此窗口. . .

8.编写一个程序，将小写字母改成大写字母；

```

#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
#include<string.h>
int main()
{
    char a[20];
    printf("请输入一个20字符内的数组:>");
    scanf("%s", a);
    int sz = strlen(a);
    int i;
    for (i = 0; i < sz; i++)
    {
        if (a[i] >= 97 && a[i] <= 122)
            a[i] = a[i] - 32;
    }
    printf("%s\n", a);
    return 0;
}

```

Microsoft Visual Studio 调试控制台

请输入一个20字符内的数组:>Kuladmir
KULADMIR

D:\Microsoft Visual 2022\Project1\x64\
按任意键关闭此窗口. . .

9.编写一个程序，让两个数的数值交换（不引入第三个变量）；
其他方法

```

#include<stdio.h>
int main()
{
    int a,b;
    scanf("%d%d",&a,&b);
    printf("a=%d b=%d\n",a,b);
    a=a+b;
    b=a-b;
    a=a-b;
    printf("a=%d b=%d\n",a,b);
    return 0;
}

```

1)

```

#include<stdio.h>
int main()
{
    int a,b;
    scanf("%d%d",&a,&b);
    printf("a=%d b=%d\n",a,b);
    a=a^b;
    b=a^b;
    a=a^b;
    printf("a=%d b=%d\n",a,b);
    return 0;
}

```

2)

如果 a,b 较大，方法 1 可能出现整型溢出的问题。方法 2 则不会。

10.编写一个猜数字游戏；

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
#include<time.h>
#include<stdlib.h>
void game()
{
    int ret = rand() % 100 + 1;
    int a, count=0;
    do
    {
        printf("请猜一个数字:>");
        scanf("%d", &a);
        if (a < ret)
        {
            printf("偏小了\n");
            count++;
        }
        else if (a > ret)
        {
            printf("偏大了\n");
            count++;
        }
        else
        {
            count++;
            printf("猜对了, 共猜对%d次\n", count);
            break;
        }
    } while (1);
}
int main()
{
    srand((size_t)time(NULL));
    game();
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
请猜一个数字:>50
偏小了
请猜一个数字:>75
偏小了
请猜一个数字:>80
偏小了
请猜一个数字:>90
偏小了
请猜一个数字:>95
偏小了
请猜一个数字:>100
偏大了
请猜一个数字:>96
偏小了
请猜一个数字:>98
偏小了
请猜一个数字:>99
猜对了, 共猜对9次
D:\Microsoft Visual 2022\Project
按任意键关闭此窗口. . .
```

11.编写一个程序，使用函数递归打印每个数的各个位数；

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
void kul(int a)
{
    if (a > 9)
        kul(a / 10);
    printf("%d ", a % 10);
}
int main()
{
    int a;
    printf("请输入一个数字\n");
    scanf("%d", &a);
    kul(a);
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
请输入一个数字
1234
1 2 3 4
D:\Microsoft Visual 2022\Project
按任意键关闭此窗口. . .
```

12.编写一个程序，求出斐波那契数列的第 n 个数；

```

#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    int a, b, c, d, e;
    printf("请输入操作数:>");
    scanf("%d", &a);
    if (a == 1 || a == 2)
        printf("第%d个数字是1\n", a);
    else
    {
        for (e = 2, b = 1, c = 1; e < a; e++)
        {
            d = b + c;
            b = c;
            c = d;
        }
        printf("第%d个数字是%d", a, d);
    }
    return 0;
}

```

```

Microsoft Visual Studio 调试控制台
请输入操作数:>4
第4个数字是3
D:\Microsoft Visual 2022\Project1\
按任意键关闭此窗口. . .

```

13.编写一个程序，不使用 strlen 函数求字符串长度；

```

#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int my_strlen(char* a)
{
    char* b = a;
    while (*b != '\0')
        b++;
    return b - a;
}
int main()
{
    char a[20];
    printf("请输入一个长度小于20的字符串:>");
    gets_s(a);
    int ret = my_strlen(a);
    printf("len = %d\n", ret);
    return 0;
}

```

```

Microsoft Visual Studio 调试控制台
请输入一个长度小于20的字符串:>kuladmir
len = 8
D:\Microsoft Visual 2022\Project1\x64\Relea
按任意键关闭此窗口. . .

```

其他方法

```

#include<stdio.h>
int strl(char* str)
{
    if(*str != '\0')
        return 1+strl(str+1);
    else
        return 0;
}
int main()
{
    char a[]="kuladmir";
    int l=strl(a);
    printf("%d\n",l);
    return 0;
}

```

1)

```

#include<stdio.h>
int strl(char* str)
{
    int c=0;
    while(*str != '\0')
    {
        c++;
        str++;
    }
    return c;
}
int main()
{
    char a[]="kuladmir";
    int l=strl(a);
    printf("%d\n",l);
    return 0;
}

```

2)

14.编写一个程序，计算一个数字的二进制补码有几个 1；

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int main()
{
    int a, count, i;
    printf("请输入一个数字:>");
    scanf("%d", &a);
    for (i = 0, count=0; i < 32; i++)
    {
        if ((a >> i) & 1 == 1)
            count++;
    }
    printf("%d 的二进制位中有 %d 个 1\n", a, count);
    return 0;
}
```

Microsoft Visual Studio 调试控制台

请输入一个数字:>-1
-1 的二进制位中有 32 个 1

D:\Microsoft Visual 2022\Project1\...
按任意键关闭此窗口。 . . .

其他方法

```
#include<stdio.h>
int count_one(int a)
{
    int count=0;
    while(a)
    {
        if(a%2 == 1)
            count++;
        a/=2;
    }
    return count;
}
int main()
{
    int a,b;
    scanf("%d",&a);
    b=count_one(a);
    printf("该数字的补码中有 %d 个 1\n",b);
    return 0;
}
```

```
#include<stdio.h>
int count_one(int a)
{
    int count=0;
    while(a)
    {
        a=a&(a-1);
        count++;
    }
    return count;
}
int main()
{
    int a,b;
    scanf("%d",&a);
    b=count_one(a);
    printf("该数字的二进制补码有 %d 个 1\n",b);
    return 0;
}
```

1) }

2) }

15.编写一个程序，判断计算机的存储方式是大端存储还是小端存储；

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
int check_sys()
{
    int a = 1;
    return *(char*)&a;
}
int main()
{
    int a=check_sys();
    if (a == 1)
        printf("小端存储\n");
    else
        printf("大端存储\n");
    return 0;
}
```

小端存储

D:\Microsoft Visual 2022\Project1\...
按任意键关闭此窗口。 . . .

16.编写一个程序，输出 100-1000 内的水仙花数；

```
int main()
{
    int i, sum, a, b, tmp;
    for (i = 100; i < 1001; i++)
    {
        tmp = i;
        b = 0;
        while (tmp)
        {
            a = tmp % 10;
            tmp = tmp / 10;
            b = a * a * a + b;
        }
        if (b == i)
            printf("%d ", i);
    }
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
153 370 371 407
D:\Microsoft Visual 2022\Project1\x64\
按任意键关闭此窗口. . .
```

17.编写一个程序，将一个字符串倒序输出；

```
#include<stdio.h>
#include<string.h>
void reverse(char* a)
{
    int b = strlen(a);
    char* left = a;
    char* right = a + b - 1;
    char tmp;
    while (left < right)
    {
        tmp = *left;
        *left = *right;
        *right = tmp;
        left++;
        right--;
    }
}

int main()
{
    char a[100];
    gets_s(a);
    reverse(a);
    printf("%s\n", a);
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
kulapati admirmoon
noomrimda itapaluk
D:\Microsoft Visual 2022\Pro
按任意键关闭此窗口. . .
```

18.编写一个程序，计算 2+22+222+2222+.....的值；

```

#define _CRT_SECURE_NO_WARNINGS 1
//函数实现Sn=2+22+222+2222+.....
#include<stdio.h>
int main()
{
    int a = 0, b, c;
    scanf("%d%d", &c, &b);
    int sum = 0, i;
    for (i = 1; i <= b; i++)
    {
        a = a * 10 + c;
        sum = sum + a;
    }
    printf("%d\n", sum);
    return 0;
}

```

```

Microsoft Visual Studio 调试控制台
2 4
2468

D:\Microsoft Visual 2022\Proj
按任意键关闭此窗口. . .

```

19.编写一个程序，左旋一个字符串；

```

#include<stdio.h>
#include<string.h>
void move(char* a, int b)
{
    int i, j, k;
    char m;
    k = strlen(a);
    for (i = 0; i < b; i++)
    {
        m = *a;
        for (j = 0; j < k - 1; j++)
            *(a + j) = *(a + j + 1);
        *(a + k - 1) = m;
    }
    puts(a);
}
int main()
{
    char a[10];
    int e, b;
    printf("请输入一个10长度内的字符串\n");
    gets_s(a);
    printf("请输入要左旋的位数:>");
    scanf("%d", &e);
    move(a, e);
    return 0;
}

```

```

Microsoft Visual Studio 调试控制台
请输入一个10长度内的字符串
kuladmir
请输入要左旋的位数:>2
ladmirku

D:\Microsoft Visual 2022\Project1\
按任意键关闭此窗口. . .

```

升级版程序：

```
#include<stdio.h>
#define N 100
unsigned int my_strlen(char* a)
{
    unsigned int count = 0;
    while (*a++ != '\0')
        count++;
    return count;
}

int main()
{
    char a[N], tmp;
    printf("请输入一个长度小于%d的字符串:>", N);
    gets_s(a);
    unsigned int len = my_strlen(a);
    printf("请输入要左旋的次数:>");
    unsigned int times, j, copy_times;
    scanf("%u", &times);
    printf("%u\n", len);
    copy_times = times;
    times = times % len;
    while (times)
    {
        tmp = *a;
        for (j = 1; j < len; j++)
            *(a + j - 1) = *(a + j);
        *(a + j - 1) = tmp;
        times--;
    }
    printf("左旋%u次的字符串为:> %s", copy_times, a);
    return 0;
}
```

Microsoft Visual Studio 调试 × + ▾

请输入一个长度小于100的字符串:>kuladmir adi
请输入要左旋的次数:>5
12
左旋5次的字符串为:> mir adikulad
C:\Users\11750\Desktop\事务文件夹\C\ConsoleApp
按任意键关闭此窗口. . .|

三步旋转法:

```
#include<stdio.h>
#include<string.h>
#include<assert.h> //断言
void change(char* a, char* b)
{
    char t;
    while (a < b)
    {
        t = *a;
        *a = *b;
        *b = t;
        a++;
        b--;
    }
}

void move(char* a, int b)
{
    assert(a != NULL); //断言函数: 如果条件为假, 则终止程序进行
    int len = strlen(a);
    assert(b <= len);
    change(a, a + b - 1);
    change(a + b, a + len - 1);
    change(a, a + len - 1);
    printf("%s", a);
}

int main()
{
    char a[20];
    int b;
    printf("请输入一个长度20以内的字符串\n");
    gets_s(a);
    printf("请输入旋转次数:>");
    scanf("%d", &b);
    move(a, b);
    return 0;
}
```

Microsoft Visual Studio 调试控制台

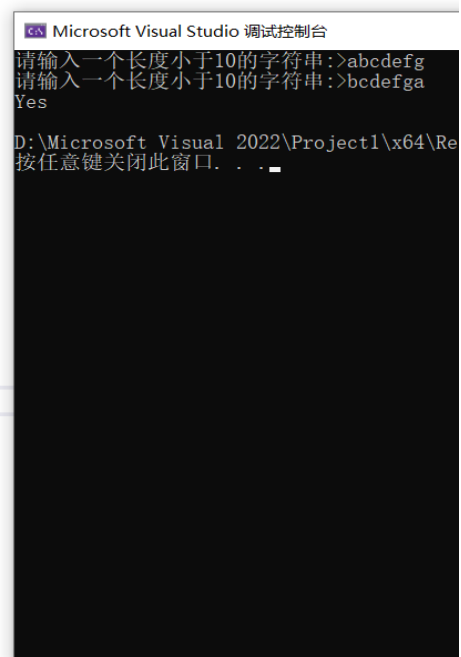
请输入一个长度20以内的字符串
kuladmir
请输入旋转次数:>2
ladmirku
D:\Microsoft Visual Studio 2022\Project1
按任意键关闭此窗口. . .

20.编写一个程序, 判断一个字符串是否由另一个旋转而来;


```

#include<stdio.h>
#include<string.h>
int main()
{
    char a[20],b[20];
    printf("请输入一个长度小于10的字符串:>");
    gets_s(a);
    printf("请输入一个长度小于10的字符串:>");
    gets_s(b);
    int len1 = strlen(a);
    int len2 = strlen(b);
    if (len1 == len2)
    {
        int sz = strlen(a);
        strncat(a, a, sz);
        char* p = strstr(a, b);
        if (p == NULL)
            printf("No\n");
        else
            printf("Yes\n");
    }
    else
        printf("No\n");
    return 0;
}

```



```

Microsoft Visual Studio 调试控制台
请输入一个长度小于10的字符串:>abcdefg
请输入一个长度小于10的字符串:>bcdefga
Yes
D:\Microsoft Visual 2022\Project1\x64\Re
按任意键关闭此窗口. . .

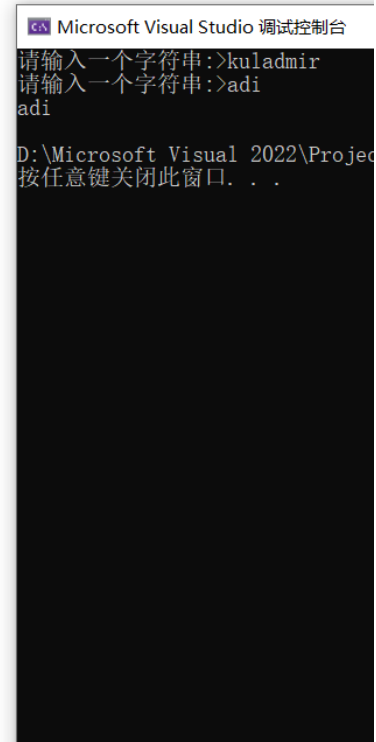
```

21.编写一个程序，实现 strcpy 函数的；

```

#define _CRT_SECURE_NO_WARNINGS 1
#include<stdio.h>
#include<assert.h>
char* my_strcpy(char* a, const char* b)
{
    assert(a != NULL); //断言函数的有效性
    assert(b != NULL);
    char* p = a;
    while (*a++ = *b++) //拷贝
    ;
    return p;
}
int main()
{
    char a[20],b[20];
    printf("请输入一个字符串:>");
    gets_s(a);
    printf("请输入一个字符串:>");
    gets_s(b);
    my_strcpy(a, b); //自定义拷贝函数
    printf("%s\n", a);
    return 0;
}

```



```

Microsoft Visual Studio 调试控制台
请输入一个字符串:>kuladmir
请输入一个字符串:>adi
adi
D:\Microsoft Visual 2022\Project1\x64\Re
按任意键关闭此窗口. . .

```

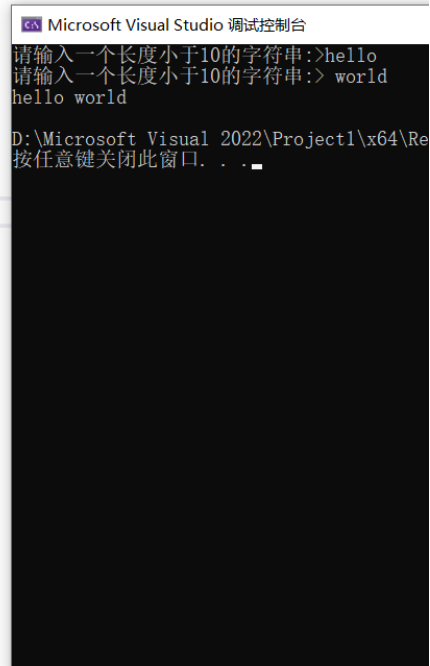
22.编写一个程序，实现 strcat 函数的；

```

#define _CRT_SECURE_NO_WARNINGS 1//实现strcat函数
#include<stdio.h>
#include<assert.h>
char* my_strcat(char* a, const char* b)
{
    assert(a != NULL);
    assert(b != NULL);
    char* p = a;
    while (*a != '\0')
    {
        a++;
    }
    while (*a++ = *b++)
    {
    }
    return p;
}

int main()
{
    char a[20], b[20];
    printf("请输入一个长度小于10的字符串:>");
    gets_s(a);
    printf("请输入一个长度小于10的字符串:>");
    gets_s(b);
    my_strcat(a, b);
    printf("%s\n", a);
    return 0;
}

```



```

Microsoft Visual Studio 调试控制台
请输入一个长度小于10的字符串:>hello
请输入一个长度小于10的字符串:> world
hello world

D:\Microsoft Visual 2022\Project1\x64\Re
按任意键关闭此窗口. . .

```

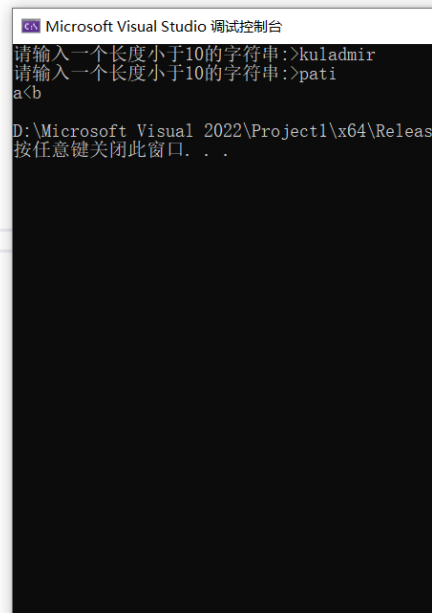
23.编写一个程序，实现 strcmp 函数；

```

#include<stdio.h>
int my_strcmp(const char* a, const char* b)
{
    while (*a == *b)
    {
        if (*a == '\0')
            return 0;
        a++;
        b++;
    }
    if (*a > *b)
        return 1;
    else
        return -1;
}

int main()
{
    char a[20], b[20];
    printf("请输入一个长度小于10的字符串:>");
    gets_s(a);
    printf("请输入一个长度小于10的字符串:>");
    gets_s(b);
    int ret = my_strcmp(a, b);
    if (ret > 0)
        printf("a>b\n");
    else if (ret < 0)
        printf("a<b\n");
    else
        printf("a==b\n");
    return 0;
}

```



```

Microsoft Visual Studio 调试控制台
请输入一个长度小于10的字符串:>kuladmir
请输入一个长度小于10的字符串:>pati
a<b

D:\Microsoft Visual 2022\Project1\x64\Releas
按任意键关闭此窗口. . .

```

20.简单界面

阅读说明：本教程中内容为部分展示，更多需要前往：

<https://docs.easyx.cn/zh-cn/intro>

1) 图形界面创建函数：**initgraph(width, height, 【flag】)**

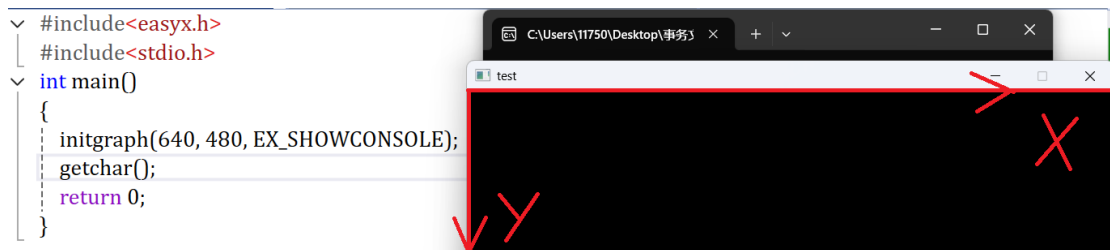
函数参数:

width: 设定了图形界面的宽度 (像素值)

height: 设定了图形界面的高度 (像素值)

flag: 设定了窗口的样式, 未指定时默认为 NULL。其可选的参数:

- EX_DBLCLKS 在绘图窗口中支持鼠标双击事件
- EX_NOCLOSE 禁用绘图窗口的关闭按钮
- EX_NOMINIMIZE 禁用绘图窗口的最小化按钮
- EX_SHOWCONSOLE 显示控制台窗口



在 **initgraph** 函数的 *flag* 部分, 如果想同时实现多个参数功能, 可以使用 | 连接多个参数。

窗口设定后, 即存在一个窗口的坐标体系。

2) 背景颜色设置函数: **setbkcolor(COLOR_NAME);**

函数参数:

COLOR_NAME: 通过指定颜色名, 以设置背景的颜色 (颜色名需要大写)

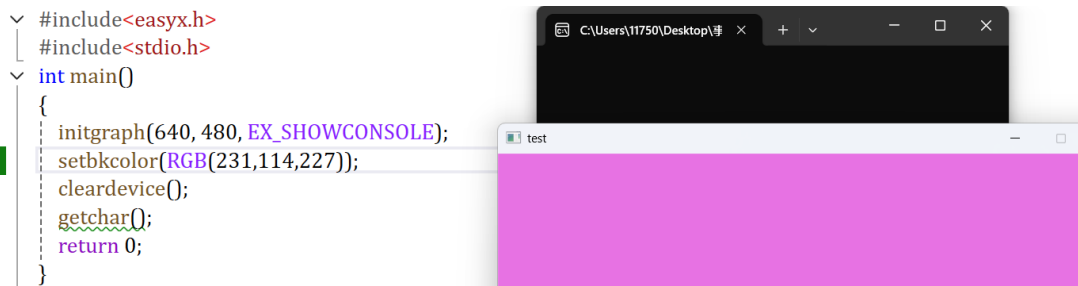
COLOR_NAME 不仅可以通过指定颜色名实现控制, 也可以使用 **RGB()** 函数完成

3) 颜色设置函数: **RGB(number_r, number_g, number_b);**

由于每个颜色都可用 RGB 三个颜色参数获取, 因此通过输入三个颜色参数, 就可以得到一个确定的颜色。该函数可以作为 **setbkcolor()** 函数的参数。

4) 清空绘图设备函数: **cleardevice();**

在使用 **setbkcolor()** 函数后, 需要在后面调用该函数, 实现颜色的填充。



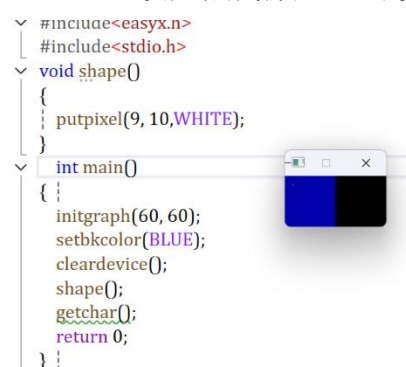
5) 点绘制函数: **putpixel(x_site, y_site, COLOR)**

函数参数:

x_site: 设置点的 x 坐

y_site: 设置点的 y 坐

COLOR: 设置点的颜色，可以使用颜色名（要大写），或者 **RGB()** 函数。



6) 线绘制函数: **line(x_init, y_init, x_fin, y_fin)**

函数参数:

提供起始点的坐标 (x_init, y_init) 和最终点的坐标 (x_fin, y_fin) 的坐标就可以绘制一条线。

最终点坐标可以 **getwidth()** 和 **getheight()** 方式，得到 **initgraph()** 函数设定的值。

6-1) 线颜色设置函数: **setlinecolor(COLOR)**

在线绘制函数前调用该函数，可以展示设定的线颜色。

6-2) 线风格设置函数:

setlinestyle(style, thickness, *<puserstyle>, <userstylecount>)

函数参数:

style: 设置线的风格。

• PS_SOLID	线为实线	—————
• PS_DASH	线为虚线	- - - - -
• PS_DOT	线为点线	
• PS_DASHDOT	线为虚线+点线	- . - . - .
• PS_DASHDOTDOT	线为虚线+点线+点线	- . . - . . - .
• PS_NULL	线不可见	
• PS_USERSTYLE	自定义线风格	

thickness: 设置线的宽（像素值）。

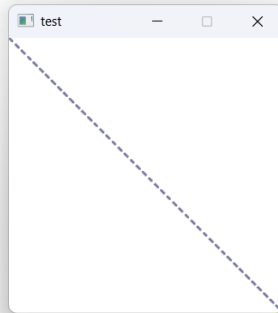
***<puserstyle>**: 当 **style=PS_USERSTYLE** 时有效。这个指针指向一个数组，数组的元素分别代表了：第一段画线长度，第一段空白长度，第二段画线长度……

<userstylecount>: 设置上述数组的元素数量。

```

#include<easyx.h>
#include<stdio.h>
void shape()
{
    setlinecolor(RGB(114,114,156));
    setlinestyle(PS_DASH, 2.5);
    line(0, 0, 256, 256);
}
int main()
{
    initgraph(256, 256, NULL);
    setbkcolor(WHITE);
    cleardevice();
    shape();
    getchar();
    return 0;
}

```



7) 矩形绘制函数:

7-1) 无填充矩形: **rectangle(x_init, y_init, x_fin, y_fin)**

函数参数:

提供起始点的坐标 (x_{init}, y_{init}) 和最终点的坐标 (x_{fin}, y_{fin}) 的坐标, 就可以获得对应的矩形。

7-2) 填充矩形: **fillrectangle(x_init, y_init, x_fin, y_fin)**

函数参数:

提供起始点的坐标 (x_{init}, y_{init}) 和最终点的坐标 (x_{fin}, y_{fin}) 的坐标, 就可以获得对应的矩形。

7-3) 无边框填充矩形: **solidrectangle(x_init, y_init, x_fin, y_fin)**

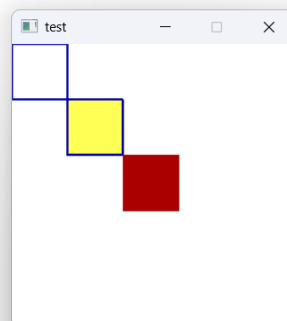
函数参数:

提供起始点的坐标 (x_{init}, y_{init}) 和最终点的坐标 (x_{fin}, y_{fin}) 的坐标, 就可以获得对应的矩形。

```

#include<easyx.h>
#include<stdio.h>
void shape()
{
    setlinecolor(BLUE);
    setlinestyle(PS_SOLID, 2);
    rectangle(0, 0, 50, 50);
    setfillcolor(YELLOW);
    fillrectangle(50, 50, 50 + 50, 50 + 50);
    setfillcolor(RED);
    solidrectangle(100, 100, 100 + 50, 150);
}
int main()
{
    initgraph(256, 256, NULL);
    setbkcolor(WHITE);
    cleardevice();
    shape();
    getchar();
    return 0;
}

```



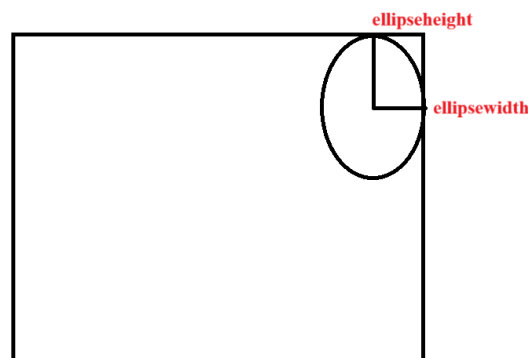
如果绘制矩形前, 没有设定线的颜色和风格, 则都会默认设定为白色。

7-4) 圆角矩形:

roundrect(*x_init*, *y_init*, *x_fin*, *y_fin*, *ellipsewidth*, *ellipseheight*)

函数参数:

提供起始点的坐标 (*x_init*, *y_init*) 和最终点的坐标 (*x_fin*, *y_fin*) 的坐标, 就可以获得对应的矩形, *ellipsewidth* 和 *ellipseheight* 可以调整圆角处的细节, 即 *ellipsewidth* 对应了圆角处参考椭圆的宽, *ellipseheight* 对应了圆角处参考椭圆的高。



7-5) 填充圆角矩形:

fillroundrect(*x_init*, *y_init*, *x_fin*, *y_fin*, *ellipsewidth*, *ellipseheight*)

函数参数:

提供起始点的坐标 (*x_init*, *y_init*) 和最终点的坐标 (*x_fin*, *y_fin*) 的坐标, 就可以获得对应的矩形, *ellipsewidth* 和 *ellipseheight* 可以调整圆角处的细节。

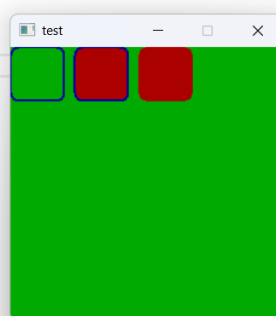
7-6) 无边框圆角矩形:

solidroundrect(*x_init*, *y_init*, *x_fin*, *y_fin*, *ellipsewidth*, *ellipseheight*)

函数参数:

提供起始点的坐标 (*x_init*, *y_init*) 和最终点的坐标 (*x_fin*, *y_fin*) 的坐标, 就可以获得对应的矩形, *ellipsewidth* 和 *ellipseheight* 可以调整圆角处的细节。

```
#include<easyx.h>
#include<stdio.h>
void shape()
{
    setlinecolor(BLUE);
    setlinestyle(PS_SOLID, 2);
    roundrect(0, 0, 50, 50, 15, 15);
    setfillcolor(RED);
    fillroundrect(60, 0, 110, 50, 15, 15);
    solidroundrect(120, 0, 170, 50, 15, 15);
}
int main()
{
    initgraph(256, 256, NULL);
    setbkcolor(GREEN);
    cleardevice();
    shape();
    getch();
    return 0;
}
```



8) 圆绘制函数

8-1) 空心圆: `round(x, y, radius)`

函数参数:

提供圆心的坐标 (x, y) 和半径, 就可以获得对应的圆形。

8-2) 实心圆: `fillround(x, y, radius)`

函数参数:

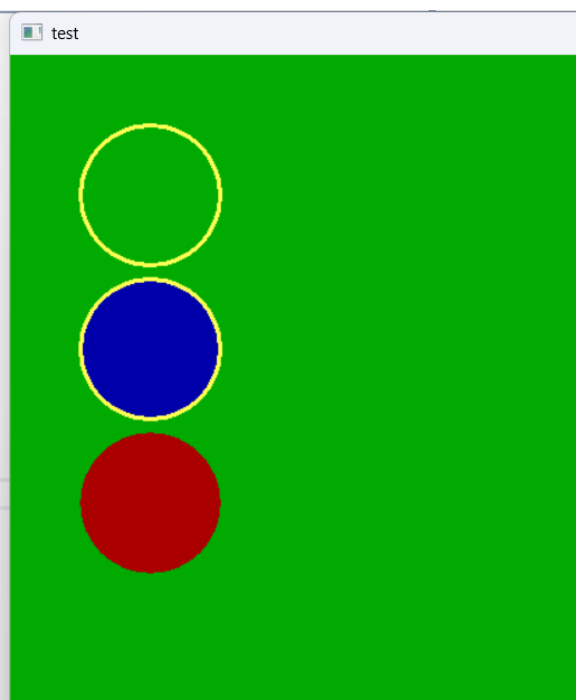
提供圆心的坐标 (x, y) 和半径, 就可以获得对应的圆形。

8-3) 无边框圆: `solidround(x, y, radius)`

函数参数:

提供圆心的坐标 (x, y) 和半径, 就可以获得对应的圆形。

```
#include<stdio.h>
#include<easyx.h>
void show()
{
    setlinecolor(YELLOW);
    setfillcolor(BLUE);
    setlinestyle(PS_SOLID, 3);
    circle(100, 100, 50);
    fillcircle(100, 210, 50);
    setfillcolor(RED);
    solidcircle(100, 320, 50);
}
int main()
{
    initgraph(640, 640);
    setbkcolor(GREEN);
    cleardevice();
    show();
    getchar();
    return 0;
}
```



9) 椭圆绘制函数

9-1) 空心椭圆: `ellipse(left, top, right, bottom)`

函数参数:

提供对角线两点 $(left, top)$ 和 $(right, bottom)$, 可以勾画一个矩形, 并限制一个与矩形四边都相切的椭圆。

9-2) 实心椭圆: `fillellipse (left, top, right, bottom)`

函数参数:

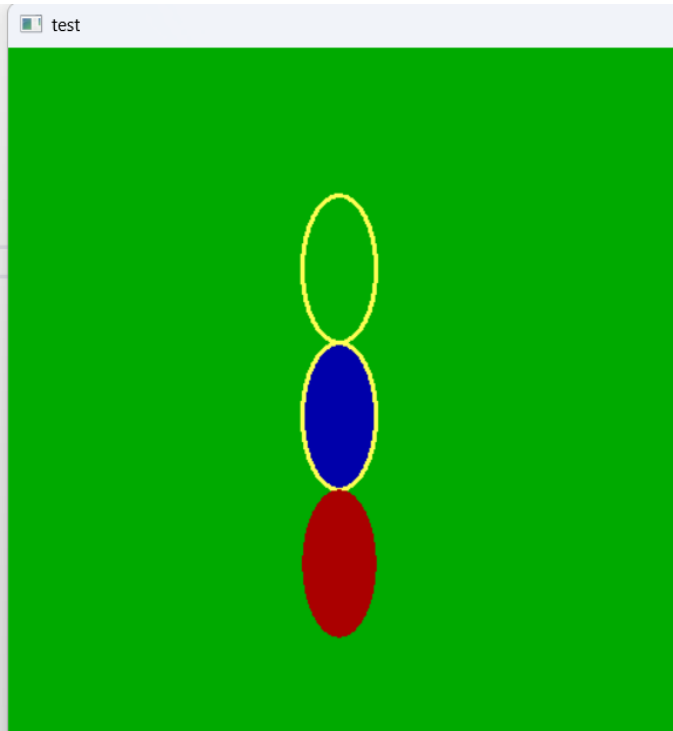
提供对角线两点 $(left, top)$ 和 $(right, bottom)$, 可以勾画一个矩形, 并限制一个与矩形四边都相切的椭圆。

9-3) 无边框椭圆: `solidellipse (left, top, right, bottom)`

函数参数:

提供对角线两点（left, top）和（right, bottom），可以勾画一个矩形，并限制一个与矩形四边都相切的椭圆。

```
#include<stdio.h>
#include<easyx.h>
void show()
{
    setlinecolor(YELLOW);
    setfillcolor(BLUE);
    setlinestyle(PS_SOLID, 3);
    ellipse(200, 100, 250, 200);
    fillellipse(200, 200, 250, 300);
    setfillcolor(RED);
    solidellipse(200, 300, 250, 400);
}
int main()
{
    initgraph(640,640);
    setbkcolor(GREEN);
    cleardevice();
    show();
    getchar();
    return 0;
}
```



更多绘制：<https://docs.easyx.cn/zh-cn/roundrect>

10) 输出文本

<注意！>由于 Easyx 中对字符的定义有 2 个字节，与一般 char 的 1 字节不匹配，因此会出现错误。

解决方法：

1. L"string";
2. _T(string)或_TEXT(string);
3. 菜单->项目->属性->高级->字符集->UNICODE 改为多字节字符集

10-1) 文本输出函数：outtextxy(x, y, _T(string))

函数参数：输入（x, y）确定文字的起始坐标，string 为输出的文字内容。

10-2) 文本颜色设置函数：settextcolor(COLOR)

在文本输出函数前调用该函数，可以展示设定的线颜色。

10-3) 文本风格设置函数：settextstyle(height, width, _T(style_name))

函数参数：

height：设置字体的大小

width：设置字体的宽度

style_name：字体风格名

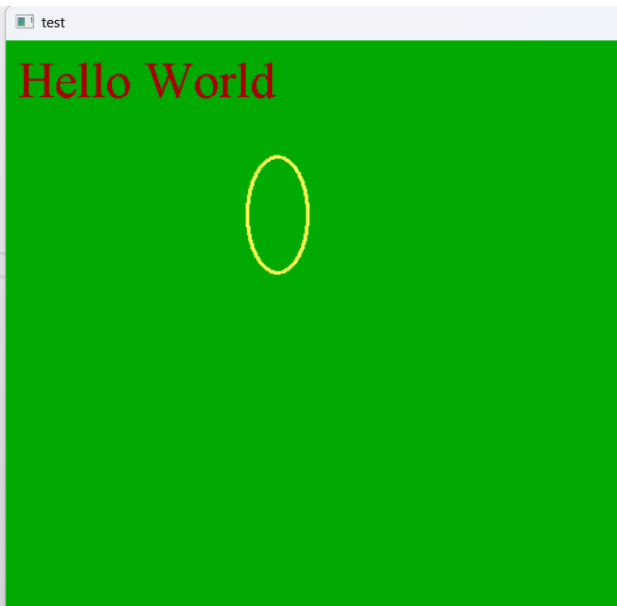
10-4) 背景模式设置函数：setbkmode(STYLE)

函数参数：

OPAQUE: 背景与界面背景相同

TRANSPARENT: 设置为透明背景

```
#include<stdio.h>
#include<easyx.h>
void show()
{
    setlinecolor(YELLOW);
    setfillcolor(BLUE);
    setlinestyle(PS_SOLID, 3);
    ellipse(200, 100, 250, 200);
    char str[50] = "Hello World";
    settextstyle(50, 0, _T("Times New Roman"));
    settextcolor(RED);
    outtextxy(10, 10, _T("Hello World"));
}
int main()
{
    initgraph(640,640);
    setbkcolor(GREEN);
    cleardevice();
    show();
    getch();
    return 0;
}
```



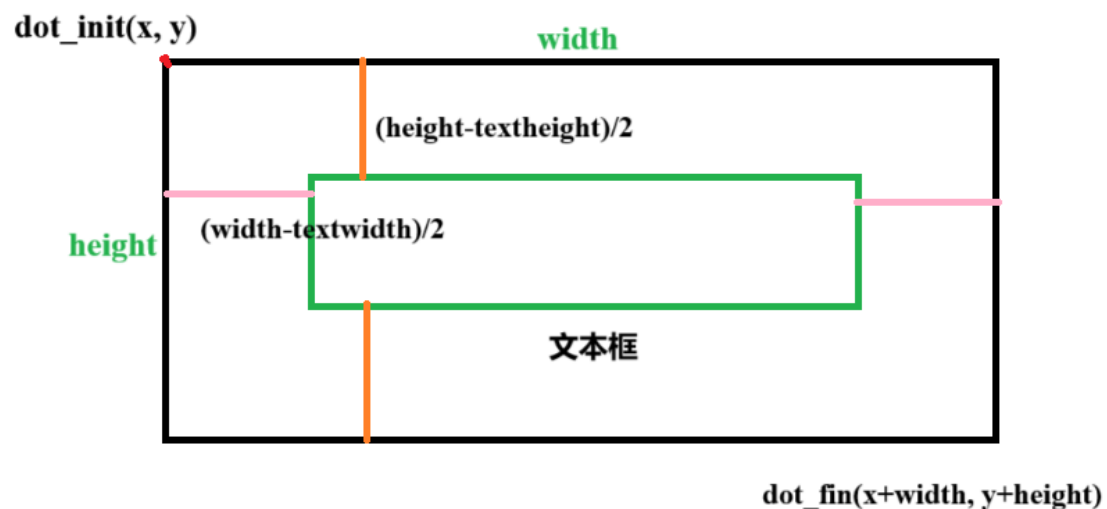
10-EX) 将文本放在矩形框中心内

```
void show()
{
    setlinecolor(YELLOW);
    setfillcolor(BLUE);
    setlinestyle(PS_SOLID, 3);
    fillrectangle(80, 150, 280, 300);
    char str[] = "Hello World";
    settextstyle(30, 0, _T("Times New Roman"));
    settextcolor(RED);
    int hSpace = (200 - textwidth(str)) / 2;
    int vSpace = (150 - textheight(str)) / 2;
    outtextxy(80 + hSpace, 150 + vSpace, str);
}
```



说明：上图中所用的为一般的矩形定义法，可以分别设置起始点 `dot_init(x,y)`，然后设置 `width` 和 `height`，这样也可以创建一个矩形。

解释：



11) 消息交互

初始化: **ExMessage msg = {0}**

这个结构体包含了许多变量 (部分):

- **message** 用于接收信号
- **x/y** 用于接收信号的位置, 需要配合 **pos(%d, %d)**
- **wheel** 用于接收滚轮滚动的方向和角度, 需要配合 **dir(%d)**

获取消息: **peekmessage(%msg, signal)**。这一函数是一个 bool 类型的, 因此需要放在判断里。signal 部分可选如下参数 (部分):

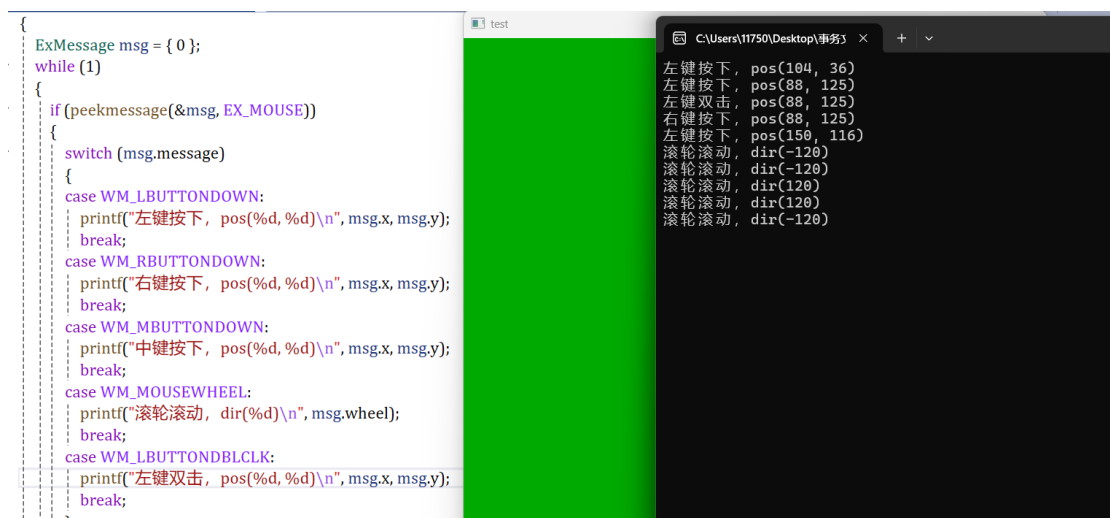
- **EX_MOUSE** 鼠标消息
- **EX_KEY** 按键消息
- **EX_WINDOW** 字符消息
- **EX_CHAR** 窗口消息

11-1) 鼠标消息

- **WM_LBUTTONDOWN** 左键按下
- **WM_RBUTTONDOWN** 右键按下
- **WM_MBUTTONDOWN** 中键按下
- **WM_MOUSEWHEEL** 滚轮滚动
- **WM_MOUSEMOVE** 鼠标移动

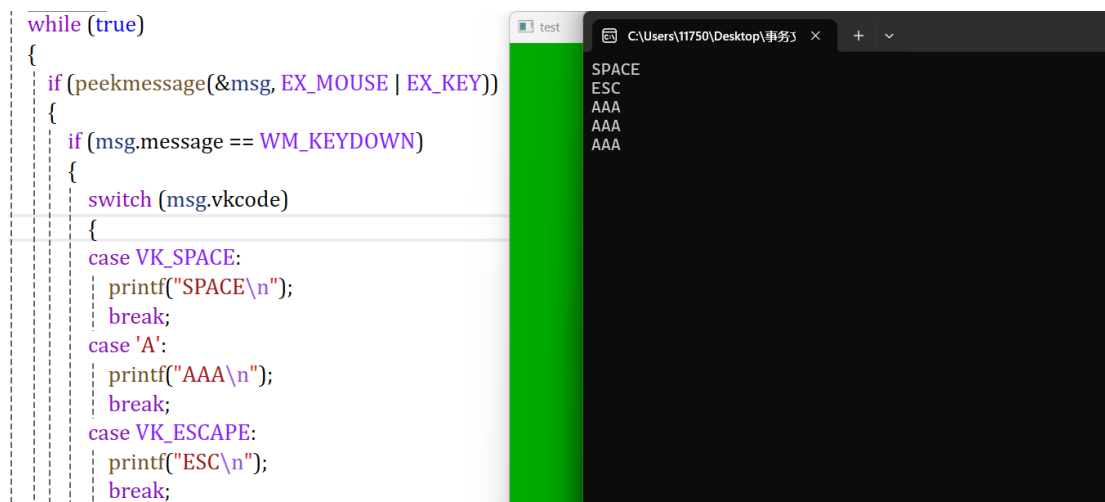
如果需要在窗口上检测到双击信号, 则 **initgraph()** 函数需要添加 **EX_DBLCLKS**

- **WM_LBUTTONDBLCLK** 左键双击
- **WM_RBUTTONDBLCLK** 右键双击
- **WM_MBUTTONDBLCLK** 中间双击



11-2) 按键信息

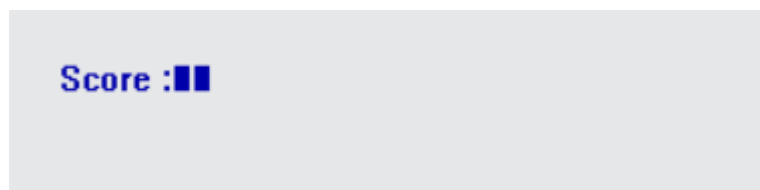
使用按键信息时, 需要在 **peekmessage()** 里额外添加一个参数: **EX_KEY**。然后可以使用 **case** 进行调用。



使用 VK 参数，可以获取例如 SPACE、ESCAPE、字符等信息，但是对于字母，只有大写才能识别。

12) 清屏、双缓冲、帧率控制

之前已经讲过，**cleardevice()**为清屏函数，可以将屏幕内容清理并填入。如果在运行时不清屏，则会产生如下结果：



因此在有数值变动的情况下，或者对背景颜色的变化的情况下，清屏是重要的。

然而，有时后会出现闪屏的情况。则需要两段函数，进行双缓冲放置闪屏。**BeginBatchDraw()**在前，**EndBatchDraw()**在后。

帧率控制中，可以控制数据的增长和变化。其原理是，通过人为设置帧率，就能够控制每一帧的执行时间。例如 1 s 设为 60 帧，则一帧即为 16.67 ms。然而有时候程序运行的时间小于 16.67 ms，则其运行完之后，使程序休眠一段时间，最终实现帧率控制。

代码实现：

```

#include<stdio.h>
#include<easyx.h>
#include<time.h> //
int main()
{
    initgraph(640, 480, EX_SHOWCONSOLE);
    setbkcolor(RGB(230, 231, 232));
    cleardevice();
    setbkmode(TRANSPARENT);

    int score = 0;
    char str[50]=" ";
    const clock_t FPS = 1000 / 60; //
    int start_time = 0; //
    int end_time = 0; //
    int fream_time = 0; //
    while (true)
    {
        start_time = clock(); //
        sprintf(str, "Score :%d", score++);
        settextcolor(BLUE);
        cleardevice();
        outtextxy(20, 20, str);
        end_time = clock(); //
        fream_time = end_time - start_time; //
        if (fream_time<FPS) //
        {
            Sleep(FPS - fream_time); //
        }
    }
    return 0;
}

```

13) 图片加载

定义图片变量：**IMAGE pic_name;**

记载图片：**loadimage(&pic_name, "path_image", width, height);**

函数参数：

&pic_name: 图片变量地址；

path_image: 图片位置，可使用绝对路径和相对路径(相对更为方便)。使用绝对路径时注意转义；

width: 图片拉伸宽度，如果没有定义，则默认为图片原宽度。（可选）

height: 图片拉伸高度，如果没有定义，则默认为图片原高度。（可选）

输出图片：**putimage(pos_x, pos_y, &pic_image, dwROP);**

函数参数：

pos_x: 图片的初始位置的 x 坐标；

pos_y: 图片的初始位置的 y 坐标；

&pic_name: 图片变量地址；

dwROP: 三元光栅操作。（可选）

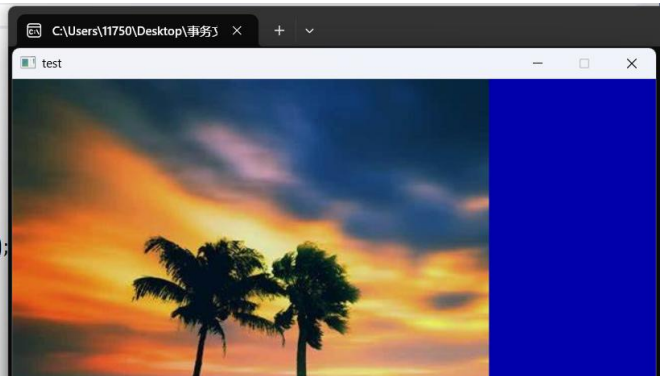
可选的三元光栅操作符（部分）：

- NOTSRCERASE 目标图像 = NOT (目标图像 OR 源图像)
- SRCCOPY 目标图像 = 源图像
- SRCERASE 目标图像 = (NOT 目标图像) AND 源图像
- DSTINVERT 目标图像 = NOT 目标图像
- MERGECOPY 目标图像 = 源图像 AND 当前填充颜色

- SRCAND

目标图像 = 目标图像 AND 源图像

```
#include<stdio.h>
#include<easyx.h>
int main()
{
    initgraph(640, 480, EX_SHOWCONSOLE);
    setbkcolor(BLUE);
    cleardevice();
    IMAGE image;
    loadimage(&image, "D:\\collect\\test.jpg");
    putimage(0, 0, &image);
    getchar();
    return 0;
}
```



13-1) 透明图贴图

在绘制一些图片时，由于其背景的原因，需要进行去背景，可以通过如下方式解决。

除了一张需要去背景的图，还需要一张其对应的掩码图。